

NSI

Terminale

Édition de travail 2026-2027

ÉDITION ÉLÈVE

01001110 01010011 01001001
01001110 01000101 01011000
01010101 01010011 00100001

NSI

Terminale

ÉDITION ÉLÈVE

Édition 2026-2027



NEXUS ♦ RÉUSSITE

Sommaire

1	1	Algorithmes sur les arbres et les graphes	1
	1.1	Taille, hauteur et parcours d'un arbre binaire	1
	1.2	Arbre binaire de recherche	2
	1.3	Parcours d'un graphe	3
	1.4	Cycle et chemin dans un graphe	4
	1.5	Diviser pour régner	5
	1.6	Programmation dynamique	6
	1.7	Recherche d'un motif : l'algorithme de Boyer-Moore	7
2	2	Architectures matérielles, systèmes d'exploitation et réseaux	15
	2.1	Le système sur puce	15
	2.2	Processus : création, ordonnancement, interblocage	16
	2.3	Protocoles de routage	17
	2.4	Chiffrement symétrique et asymétrique	18
3	3	Bases de données relationnelles	23
	3.1	Le modèle relationnel	23
	3.2	Le système de gestion de bases de données	24
	3.3	Interroger une base : SELECT, FROM, WHERE	25
	3.4	Croiser les données : JOIN et ORDER BY	25
	3.5	Modifier une base : INSERT, UPDATE, DELETE	26
4	4	Histoire de l'informatique	33
	4.1	Événements clés de l'histoire de l'informatique	33
	4.2	Évolution des rôles du logiciel et du matériel	33
5	5	Langages, récursivité et paradigmes de programmation	37
	5.1	Programme, donnée et calculabilité	37
	5.2	Écrire et analyser un programme récursif	38
	5.3	API, bibliothèques et modules	39
	5.4	Paradigmes de programmation	40
	5.5	Mise au point : causes typiques de bugs	41
6	6	Structures de données	47
	6.1	Interface et implémentation d'une structure de données	47
	6.2	Définir et utiliser une classe en Python	48

		◆
6.3	Piles, files, et choix de structure adaptée	49
6.4	Arbres binaires	50
6.5	Graphes : modélisation et représentations	51
7	7 Conduire un projet informatique	59
7.1	Conduire un projet informatique en Terminale	59

1 Algorithmes sur les arbres et les graphes

1.1 Taille, hauteur et parcours d'un arbre binaire

On représente un arbre binaire par des nœuds chaînés :

```
1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
```

Un arbre vide est représenté par None.

DÉFINITION 1

La **taille** d'un arbre est son nombre de nœuds. La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille : par convention, la hauteur de l'arbre vide est -1 , et la hauteur d'une feuille (nœud sans enfant) est 0 .

Exemple.

```
1 def taille(arbre):
2     if arbre is None:
3         return 0
4     return 1 + taille(arbre.gauche) + taille(arbre.droite)
5
6 def hauteur(arbre):
7     if arbre is None:
8         return -1
9     return 1 + max(hauteur(arbre.gauche), hauteur(arbre.droite))
```

◆ **PROPRIÉTÉ Parcours en profondeur** Un parcours **en profondeur** visite récursivement les sous-arbres avant de « remonter ». Trois ordres sont usuels selon la position de la racine :

- ▶ **préfixe** : racine, puis sous-arbre gauche, puis sous-arbre droit ;
- ▶ **infixe** : sous-arbre gauche, puis racine, puis sous-arbre droit ;
- ▶ **suffixe** : sous-arbre gauche, puis sous-arbre droit, puis racine.

Exemple.

```
1 def infixe(arbre):
2     if arbre is None:
3         return []
4     return infixe(arbre.gauche) + [arbre.valeur] + infixe(arbre.droite)
```

DÉFINITION 2

Un parcours **en largeur** (*Breadth-First Search*, BFS) visite les nœuds **niveau par niveau**, de la racine vers les feuilles. Il s'implémente avec une **file** : on enfile la racine, puis, tant que la file n'est pas vide, on défile un nœud, on le traite, et on enfile ses enfants non vides.

```

1 def largeur(arbre):
2     if arbre is None:
3         return []
4     resultat = []
5     file = [arbre]
6     while file:
7         noeud = file.pop(0)
8         resultat.append(noeud.valeur)
9         if noeud.gauche is not None:
10            file.append(noeud.gauche)
11        if noeud.droite is not None:
12            file.append(noeud.droite)
13    return resultat

```

⚠ ERREUR FRÉQUENTE

Confondre parcours en profondeur et en largeur. Un parcours en profondeur (préfixe/infixe/suffixe) descend au maximum dans un sous-arbre avant de passer au suivant ; un parcours en largeur traite **tous** les nœuds d'un niveau avant de passer au niveau suivant. Sur l'arbre Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3)), le parcours préfixe donne [1, 2, 4, 5, 3] alors que le parcours en largeur donne [1, 2, 3, 4, 5].

1.2 Arbre binaire de recherche

DÉFINITION 1

Un **arbre binaire de recherche** (ABR) est un arbre binaire dans lequel, pour chaque nœud, toutes les valeurs du sous-arbre gauche sont **inférieures** à la valeur du nœud, et toutes celles du sous-arbre droit lui sont **supérieures**.

Exemple. Rechercher une clé dans un ABR : on compare la clé cherchée à la racine, et on descend à gauche ou à droite selon le résultat.

```

1 class Noeud:
2     def __init__(self, valeur, gauche=None, droite=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droite = droite
6
7 def rechercher(arbre, cle):
8     if arbre is None:
9         return False
10    if cle == arbre.valeur:
11        return True
12    if cle < arbre.valeur:
13        return rechercher(arbre.gauche, cle)
14    return rechercher(arbre.droite, cle)

```

◆ **PROPRIÉTÉ Coût de la recherche** À chaque comparaison, la recherche élimine tout un sous-arbre : le coût de la recherche est proportionnel à la **hauteur** de l'arbre. Pour un arbre équilibré à n nœuds, cette hauteur est de l'ordre de $\log_2 n$: la recherche est très rapide. Pour un arbre dégénéré (ressemblant à une liste chaînée), elle peut coûter jusqu'à n comparaisons.

DÉFINITION 2

Insérer une clé dans un ABR consiste à descendre dans l'arbre comme pour une recherche, jusqu'à atteindre un sous-arbre vide, et à y placer la nouvelle clé sous forme de feuille : cela préserve la propriété d'ABR.

```
1 def inserer(arbre, cle):
2     if arbre is None:
3         return Noeud(cle)
4     if cle < arbre.valeur:
5         arbre.gauche = inserer(arbre.gauche, cle)
6     elif cle > arbre.valeur:
7         arbre.droite = inserer(arbre.droite, cle)
8     return arbre
```

⚠ ERREUR FRÉQUENTE

Oublier de réaffecter le résultat de inserer. Écrire simplement `inserer(arbre.gauche, cle)` sans faire `arbre.gauche = inserer(arbre.gauche, cle)` ne modifie rien lorsque `arbre.gauche` vaut `None` : un nouveau nœud est bien créé et renvoyé par la fonction, mais il n'est jamais rattaché à l'arbre existant.

1.3 Parcours d'un graphe

On représente un graphe par un dictionnaire associant à chaque sommet la liste de ses successeurs (voir le chapitre *Structures de données*).

DÉFINITION 1

Le parcours en **largeur d'abord** (*Breadth-First Search*, BFS) d'un graphe visite les sommets par distance croissante au sommet de départ, en utilisant une **file**. Contrairement à un arbre, un graphe peut contenir des **cycles** : il faut donc mémoriser les sommets déjà visités pour ne pas boucler indéfiniment.

Exemple.

```
1 def bfs(graphe, depart):
2     visites = [depart]
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         for voisin in graphe[sommet]:
7             if voisin not in visites:
8                 visites.append(voisin)
9                 file.append(voisin)
```

```
10 return visites
```

DÉFINITION 2

Le parcours en **profondeur d'abord** (*Depth-First Search*, DFS) explore chaque branche aussi loin que possible avant de revenir en arrière (*backtrack*). Il s'implémente naturellement par **réursion** (la pile d'appels joue le rôle de la pile), ou explicitement avec une **pile**.

Exemple. Version récursive :

```
1 def dfs(graphe, sommet, visites=None):
2     if visites is None:
3         visites = []
4         visites.append(sommet)
5     for voisin in graphe[sommet]:
6         if voisin not in visites:
7             dfs(graphe, voisin, visites)
8     return visites
```

⚠ ERREUR FRÉQUENTE

Argument par défaut mutable. Écrire `def dfs(graphe, sommet, visites=[])` : au lieu de `visites=None` est un piège classique en Python : la liste par défaut est créée **une seule fois**, à la définition de la fonction, et **partagée** entre tous les appels qui ne fournissent pas explicitement `visites`. Un deuxième appel `dfs(graphe, 0)` repartirait alors avec les sommets déjà visités lors du premier appel.

◆ **PROPRIÉTÉ BFS et DFS : deux usages différents** Le BFS trouve le **plus court chemin** en nombre d'arêtes dans un graphe non pondéré (il visite par distance croissante). Le DFS est souvent plus simple à écrire et suffit pour explorer entièrement un graphe ou détecter sa connexité, mais ne garantit pas de trouver le chemin le plus court.

1.4 Cycle et chemin dans un graphe

DÉFINITION 1

Un **cycle** est un chemin qui part d'un sommet et y revient sans repasser deux fois par la même arête. Pour un graphe **non orienté**, on détecte un cycle lors d'un parcours (BFS ou DFS) si l'on rencontre un sommet déjà visité qui n'est **pas** le sommet dont on vient directement (son parent dans le parcours).

Exemple.

```
1 def a_un_cycle(graphe, sommet, visites, parent):
2     visites.append(sommet)
3     for voisin in graphe[sommet]:
4         if voisin not in visites:
5             if a_un_cycle(graphe, voisin, visites, sommet):
6                 return True
7     elif voisin != parent:
```

```

8     return True
9     return False

```

DÉFINITION 2

Chercher un chemin entre deux sommets A et B revient à effectuer un parcours (BFS ou DFS) depuis A en mémorisant, pour chaque sommet visité, le sommet depuis lequel il a été atteint ; on remonte ensuite cette chaîne depuis B pour reconstituer le chemin. **L'absence de chemin** doit être explicitement traitée : si B n'est jamais atteint, aucun chemin n'existe.

```

1 def chemin(graphe, depart, arrivee):
2     predecesseur = {depart: None}
3     file = [depart]
4     while file:
5         sommet = file.pop(0)
6         if sommet == arrivee:
7             break
8         for voisin in graphe[sommet]:
9             if voisin not in predecesseur:
10                predecesseur[voisin] = sommet
11                file.append(voisin)
12     if arrivee not in predecesseur:
13         return None
14     chemin_trouve = []
15     sommet = arrivee
16     while sommet is not None:
17         chemin_trouve.append(sommet)
18         sommet = predecesseur[sommet]
19     chemin_trouve.reverse()
20     return chemin_trouve

```

⚠ ERREUR FRÉQUENTE

Ne pas traiter le cas où aucun chemin n'existe. Si l'on oublie de tester `if arrivee not in predecesseur`, la reconstruction du chemin lève une `KeyError` lorsque le sommet d'arrivée n'a jamais été atteint (graphe non connexe). Un algorithme de recherche de chemin correct doit toujours prévoir ce cas et renvoyer une valeur explicite (ici `None`) plutôt que de planter.

1.5 Diviser pour régner

DÉFINITION 1

La méthode **diviser pour régner** résout un problème en trois étapes : **diviser** l'instance en sous-instances plus petites de même nature, **régner** en résolvant récursivement chaque sous-instance, puis **combinaison** des solutions partielles pour obtenir la solution du problème initial.

Exemple.

```

1 def fusionner(gauche, droite):
2     resultat = []
3     i, j = 0, 0
4     while i < len(gauche) and j < len(droite):

```



```
5     if gauche[i] <= droite[j]:
6         resultat.append(gauche[i])
7         i += 1
8     else:
9         resultat.append(droite[j])
10        j += 1
11    return resultat + gauche[i:] + droite[j:]
12
13 def tri_fusion(liste):
14     if len(liste) <= 1:
15         return liste
16     milieu = len(liste) // 2
17     gauche = tri_fusion(liste[:milieu])
18     droite = tri_fusion(liste[milieu:])
19     return fusionner(gauche, droite)
```

◆ **PROPRIÉTÉ Coût du tri fusion** À chaque niveau de récursion, la liste est divisée en deux, ce qui donne $\log_2 n$ niveaux ; à chaque niveau, la fusion de toutes les sous-listes coûte n comparaisons au total. Le coût total du tri fusion est donc de l'ordre de $n \log_2 n$, bien meilleur que les n^2 comparaisons d'un tri par sélection ou par insertion sur de grandes listes.

⚠ ERREUR FRÉQUENTE

Oublier le cas de base. Sans le test `if len(liste) <= 1: return liste`, la récursion ne s'arrête jamais : diviser une liste vide donnerait deux listes vides, et l'appel récursif se répéterait indéfiniment (ou lèverait une erreur de profondeur de récursion maximale atteinte).

+ POUR ALLER PLUS LOIN

La **rotation d'image** est un autre exemple classique de diviser pour régner : on découpe l'image en quadrants, on fait pivoter chaque quadrant récursivement, puis on réassemble le résultat en permutant les quadrants entre eux.

1.6 Programmation dynamique

DÉFINITION 1

La **programmation dynamique** résout un problème en le décomposant en sous-problèmes, comme diviser pour régner, mais s'applique lorsque ces sous-problèmes se **recoupent** (les mêmes sous-problèmes réapparaissent plusieurs fois). On **mémore** (*mémoïsation*) le résultat de chaque sous-problème déjà résolu pour ne jamais le recalculer.

Exemple. Version naïve, sans mémoïsation : recalcule plusieurs fois les mêmes sous-sommes.

```
1 def rendu_naif(pieces, somme):
2     if somme == 0:
3         return 0
4     meilleur = None
5     for p in pieces:
6         if p <= somme:
7             candidat = 1 + rendu_naif(pieces, somme - p)
```

```

8         if meilleur is None or candidat < meilleur:
9             meilleur = candidat
10    return meilleur

```

◆ **PROPRIÉTÉ Mémoïsation** On stocke chaque résultat déjà calculé dans un **dictionnaire** (somme → nombre minimal de pièces) : avant de calculer, on vérifie si le résultat est déjà connu ; sinon, on le calcule puis on l'enregistre avant de le renvoyer.

```

1 def rendu_memo(pieces, somme, memo=None):
2     if memo is None:
3         memo = {0: 0}
4     if somme in memo:
5         return memo[somme]
6     meilleur = None
7     for p in pieces:
8         if p <= somme:
9             candidat = 1 + rendu_memo(pieces, somme - p, memo)
10            if meilleur is None or candidat < meilleur:
11                meilleur = candidat
12    memo[somme] = meilleur
13    return meilleur

```

⚠ ERREUR FRÉQUENTE

Argument memo par défaut mutable partagé entre appels. Comme pour un graphe, écrire `memo={0: 0}` directement dans la signature créerait un dictionnaire partagé et corrompu entre appels indépendants ; on utilise `memo=None` puis on l'initialise à l'intérieur de la fonction.

◆ **PROPRIÉTÉ Gain apporté par la mémoïsation** Sans mémoïsation, le nombre d'appels récurifs croît de façon **exponentielle** avec la somme à rendre (les mêmes sous-sommes sont recalculées un grand nombre de fois). Avec mémoïsation, chaque sous-somme n'est calculée **qu'une seule fois** : le coût devient proportionnel au nombre de sous-problèmes distincts (ici, la valeur de la somme), au prix d'un espace mémoire supplémentaire pour stocker le dictionnaire.

1.7 Recherche d'un motif : l'algorithme de Boyer-Moore

DÉFINITION 1

Rechercher un **motif** (chaîne courte) dans un **texte** (chaîne longue) consiste à trouver la ou les positions où le motif apparaît dans le texte. La méthode **naïve** teste, pour chaque position du texte, si le motif y correspond caractère par caractère.

```

1 def recherche_naive(texte, motif):
2     positions = []
3     for i in range(len(texte) - len(motif) + 1):
4         if texte[i:i + len(motif)] == motif:
5             positions.append(i)
6     return positions

```

◆ **PROPRIÉTÉ Principe de Boyer-Moore** L'algorithme de Boyer-Moore compare le motif au texte de **droite à gauche**, et se déplace en s'appuyant sur un **prétraitement** du motif : lorsqu'un caractère du texte ne correspond pas, on utilise la position de ce caractère dans le motif (règle du « mauvais caractère ») pour décaler le motif directement à la position suivante où une correspondance est encore possible, au lieu d'avancer d'une seule case comme la méthode naïve.

Exemple. Règle simplifiée du mauvais caractère (décalage fondé sur la dernière occurrence de chaque caractère dans le motif) :

```

1 def table_decalage(motif):
2     table = {}
3     for i, c in enumerate(motif):
4         table[c] = i
5     return table
6
7 def boyer_moore_simplifie(texte, motif):
8     table = table_decalage(motif)
9     n, m = len(texte), len(motif)
10    positions = []
11    i = 0
12    while i <= n - m:
13        j = m - 1
14        while j >= 0 and texte[i + j] == motif[j]:
15            j -= 1
16        if j < 0:
17            positions.append(i)
18            i += 1
19        else:
20            decalage_c = j - table.get(texte[i + j], -1)
21            i += max(1, decalage_c)
22    return positions

```

⚠ ERREUR FRÉQUENTE

Croire que Boyer-Moore compare toujours de gauche à droite comme la méthode naïve. C'est précisément l'inverse : c'est en comparant le motif au texte **de droite à gauche**, tout en avançant le motif de gauche à droite le long du texte, que l'algorithme peut exploiter un désaccord précoce (sur le dernier caractère comparé) pour décaler le motif de plusieurs positions d'un coup.

+ POUR ALLER PLUS LOIN

Le programme ne demande pas l'étude formelle du coût de Boyer-Moore, qui est délicate (le meilleur cas est sous-linéaire, mais certaines variantes du pire cas restent quadratiques sans une règle de décalage supplémentaire, dite du « bon suffixe », non traitée ici).

④ MÉTHODE M1 Dérouler à la main les parcours d'un arbre binaire

Quand l'utiliser ? Quand l'énoncé donne un arbre **dessiné** et demande la liste des valeurs dans l'ordre préfixe, infixe, suffixe, ou en largeur. Le code du cours ne t'aide pas ici : c'est toi la machine, et c'est l'ordre de tes gestes qui décide du résultat.

Pas à pas — les trois parcours en profondeur :

1. **Choisis ta place autour du nœud.** Préfixe : on écrit la racine **avant** de descendre. Infixe : on écrit la racine **entre** les deux descentes. Suffixe : on écrit la racine **après** les deux descentes.
2. **Descends toujours à gauche d'abord**, jusqu'à un sous-arbre vide. Un sous-arbre vide ne produit rien : on remonte immédiatement.
3. **Remonte et applique la règle de l'étape 1** au nœud d'où tu viens.
4. **Descends ensuite à droite**, avec la même règle, jusqu'à épuisement.
5. **Compte tes valeurs.** La liste finale doit contenir **exactement** autant de valeurs que l'arbre a de nœuds : c'est le premier contrôle à faire.

Pas à pas — le parcours en largeur : tiens un tableau à deux colonnes, l'état de la file et la valeur qu'on sort.

1. **Enfile la racine.**
2. **Défile le nœud de tête**, écris sa valeur dans le résultat.
3. **Enfile ses enfants non vides**, gauche puis droite, à la **fin** de la file.
4. **Recommence** tant que la file n'est pas vide.

Exemple : l'arbre de racine 1, de fils gauche 2 (lui-même de fils 4 et 5) et de fils droit 3.

Préfixe : on écrit 1, on descend à gauche, on écrit 2, on descend à gauche, on écrit 4 (pas d'enfants), on remonte, on descend à droite, on écrit 5, on remonte deux fois, on descend à droite, on écrit 3. Résultat : 1, 2, 4, 5, 3.

Infixe : la même descente, mais la racine s'écrit entre les deux : 4, 2, 5, 1, 3.

Suffixe : la racine après les deux : 4, 5, 2, 3, 1.

Largeur : file [1] ; on sort 1, on enfile 2 et 3 ; file [2, 3] ; on sort 2, on enfile 4 et 5 ; file [3, 4, 5] ; on sort 3, 4, 5. Résultat : 1, 2, 3, 4, 5.

Pièges :

- **Confondre préfixe et largeur.** Sur un arbre en peigne, les deux donnent la même liste ; dès que l'arbre se ramifie, elles diffèrent. Ne conclus jamais d'un exemple dégénéré.
- **Écrire la racine trop tôt en infixe.** L'infixe n'écrit la racine qu'une fois le sous-arbre gauche entièrement terminé, pas au moment où on l'atteint.
- **Enfiler les enfants au début de la file.** Une file se remplit par la fin : les enfileur au début en ferait une pile, et tu obtiendrais un parcours en profondeur.
- **Oublier un sous-arbre vide.** Un nœud à un seul enfant a bien un côté vide, qui ne produit rien mais ne dispense pas de remonter.

Vérifier : compte les valeurs — il en faut autant que de nœuds, sans répétition. Et si l'arbre est un arbre binaire de recherche, l'infixe doit sortir **croissant** : c'est le contrôle le plus rapide qui existe.

S'entraîner : exercices C3 et C4 du chapitre.

④ MÉTHODE M2 Dérouler à la main un parcours de graphe

Quand l'utiliser ? Quand l'énoncé donne un graphe et demande l'ordre de visite des sommets en largeur ou en profondeur. Contrairement à un arbre, un graphe a des cycles : sans mémoire des sommets déjà vus, tu tournes en rond. Deux états sont donc à tenir, pas un.

Pas à pas :

1. **Fixe l'ordre des voisins** et écris-le. C'est presque toujours l'ordre du dictionnaire donné par l'énoncé. Sans cette convention, deux réponses différentes sont également défendables, et la correction ne peut pas trancher.

2. **Prépare deux colonnes** : la structure d'attente, et la liste des visités. Largeur : une **file**, on prend au début, on ajoute à la fin. Profondeur : une **pile**, on prend et on ajoute du même côté — ou la récursion, qui est la même pile.
3. **Marque le sommet de départ comme visité** au moment où tu l'ajoutes, pas au moment où tu le sors. Marquer trop tard laisse un sommet entrer deux fois.
4. **Sors un sommet, écris-le dans le résultat**, puis passe ses voisins en revue **dans l'ordre fixé** : tout voisin non visité est marqué et ajouté.
5. **Recommence** tant que la structure n'est pas vide.
6. **Si des sommets restent non visités**, c'est que le graphe n'est pas connexe : l'énoncé demande alors de relancer depuis un sommet non visité, ou de conclure.

Exemple : $G = \{0 : [1, 2], 1 : [0, 3], 2 : [0, 3], 3 : [1, 2]\}$, départ 0.

En largeur : visités $\{0\}$, file $[0]$. On sort 0, voisins 1 puis 2 : les deux sont neufs, on les marque et on les enfile. File $[1, 2]$. On sort 1, voisins 0 (déjà vu) et 3 (neuf, marqué, enfilé). File $[2, 3]$. On sort 2 : 0 et 3 sont déjà vus. On sort 3. Résultat : 0, 1, 2, 3.

En profondeur (récursive) : on visite 0, on part sur 1, de 1 on part sur 3, de 3 on part sur 2 — car 1 est déjà visité — et de 2 tout est vu. Résultat : 0, 1, 3, 2.

Les deux listes contiennent les mêmes sommets et **pas** dans le même ordre : c'est normal, et c'est même tout l'intérêt de la question.

Pièges :

- ▶ **Ne pas fixer l'ordre des voisins**. L'ordre de visite en dépend entièrement : en inversant les listes du même graphe, le parcours en largeur donne 0, 2, 1, 3 au lieu de 0, 1, 2, 3.
- ▶ **Marquer un sommet au moment où on le sort**. Il a alors pu être ajouté plusieurs fois entre-temps. L'ordre final peut rester bon, mais la structure gonfle, et sur un graphe dense la copie manuscrite devient fausse.
- ▶ **Oublier la mémoire des visités**. Le cycle $0 - 1 - 3 - 2 - 0$ suffit à faire boucler l'algorithme indéfiniment.
- ▶ **Croire qu'un parcours visite tout**. Il ne visite que la composante connexe du départ. Les sommets restants doivent être mentionnés.

Vérifier : chaque sommet apparaît **une seule fois** dans le résultat, et le premier est le sommet de départ. En largeur, contrôle en plus que les sommets sortent par distance croissante au départ : d'abord les voisins directs, puis leurs voisins.

S'entraîner : exercices C7 et C8 du chapitre.

🔗 MÉTHODE M3 Structurer un algorithme diviser pour régner

Quand l'utiliser ? Quand on te demande d'écrire un algorithme « en utilisant la méthode diviser pour régner » sur un problème que le cours n'a pas traité. Le tri fusion ne te servira pas de modèle à recopier : c'est sa **structure** qu'il faut retrouver sur ton problème.

Pas à pas :

1. **Écris d'abord le cas de base**, avant tout le reste. Quelle est la plus petite instance que tu sais résoudre sans rien découper ? Une liste vide, une liste à un élément, un intervalle réduit à un point. Sans ce cas, la récursion ne s'arrête jamais.
2. **Dis comment tu divises**, et vérifie que les sous-instances sont **strictement plus petites**. Couper en deux moitiés, retirer un élément, diviser l'exposant par deux : peu importe, pourvu que la taille décroisse à chaque appel.
3. **Formule l'appel récursif** en supposant qu'il fonctionne. C'est l'étape qui demande de la confiance : tu n'as pas à dérouler la récursion, tu as à écrire ce que tu ferais si la réponse aux sous-problèmes t'était donnée.
4. **Écris la combinaison**. Comment reconstitue-t-on la réponse du problème entier à partir des réponses partielles ? C'est là que se trouve le vrai travail — fusionner deux listes triées, additionner deux comptes, prendre un maximum.

5. **Vérifie la couverture des cas.** Le cas de base et le cas général doivent couvrir toutes les tailles possibles, sans trou ni recouvrement.
6. **Estime le coût :** combien de sous-problèmes par appel, de quelle taille, et combien coûte la combinaison. Deux moitiés plus une combinaison linéaire donnent $n \log_2 n$.

Exemple : compter les occurrences d'une valeur dans une liste, par diviser pour régner.

Cas de base : une liste vide contient 0 occurrence ; une liste à un élément en contient 1 si c'est la valeur cherchée, 0 sinon.

Division : on coupe en deux moitiés au milieu ; chacune est strictement plus courte dès que la liste a au moins deux éléments.

Récursion : on suppose savoir compter dans chaque moitié.

Combinaison : le compte total est la **somme** des deux comptes — aucune occurrence n'est à cheval, puisqu'une occurrence est un élément et qu'un élément est dans une moitié ou dans l'autre.

Coût : deux sous-problèmes de taille $n/2$ et une combinaison en temps constant, donc un coût proportionnel à n — le même qu'un parcours simple. Diviser pour régner ne rend pas tout plus rapide, et le dire fait partie de la réponse.

Pièges :

- **Écrire la division avant le cas de base.** C'est l'ordre qui fait oublier le cas de base, et une récursion sans cas de base ne termine pas : couper une liste vide en deux donne deux listes vides, indéfiniment.
- **Une division qui ne fait pas décroître la taille.** Couper « la liste et elle-même », ou traiter n puis n , boucle. La décroissance stricte est ce qui garantit d'atteindre le cas de base.
- **Vouloir dérouler la récursion dans sa tête.** On écrit un algorithme récursif en supposant les appels corrects, pas en simulant trois niveaux.
- **Croire que diviser pour régner accélère toujours.** Ici le coût reste linéaire. Le gain du tri fusion vient de ce que la combinaison remplace un travail quadratique, pas du découpage lui-même.

Vérifier : teste ton algorithme sur les trois tailles qui cassent tout — 0, 1, et 2 éléments — puis compare-le à une méthode directe sur une dizaine de cas au hasard. Si les deux divergent une seule fois, c'est la combinaison qu'il faut relire.

S'entraîner : exercices C11 du chapitre.

🔗 MÉTHODE M4 Passer d'une récursion naïve à une version mémorisée

Quand l'utiliser ? Quand l'énoncé demande de « résoudre par programmation dynamique » ou signale qu'une fonction récursive est « trop lente ». La programmation dynamique n'est pas un nouvel algorithme à inventer : c'est une transformation mécanique d'un algorithme récursif que tu as déjà.

Pas à pas :

1. **Écris d'abord la version naïve**, même si tu sais qu'elle sera lente. Sans elle, tu n'as rien à transformer, et c'est elle qui porte la logique du problème.
2. **Vérifie que les sous-problèmes se RECOUPENT.** Déroule deux niveaux d'appels : si le même argument revient dans deux branches différentes, la mémorisation servira. S'il ne revient jamais — comme dans le tri fusion — elle ne gagnerait rien.
3. **Identifie la clé :** quel argument détermine entièrement le résultat ? C'est lui qui indexera le dictionnaire. Si deux arguments varient, la clé est le couple.
4. **Ajoute le dictionnaire en paramètre**, avec la valeur par défaut None, et initialise-le à l'intérieur de la fonction. Jamais `memo={}` dans la signature.
5. **Encadre le corps de deux lignes :** au début, « si la clé est connue, renvoie la valeur mémorisée » ; à la fin, « enregistre le résultat avant de le renvoyer ».
6. **Contrôle que le résultat n'a pas changé** sur plusieurs valeurs. La mémorisation accélère ; elle ne doit rien calculer d'autre.

Exemple : la suite de Fibonacci.

Version naïve : $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, avec $\text{fib}(0)=0$ et $\text{fib}(1)=1$.

Recoupement : $\text{fib}(5)$ appelle $\text{fib}(4)$ et $\text{fib}(3)$; mais $\text{fib}(4)$ rappelle $\text{fib}(3)$. Le même sous-problème est calculé deux fois, et ce doublement se répète à chaque niveau.

Clé : l'entier n suffit.

Après transformation, $\text{fib}(30)$ demande 31 calculs distincts au lieu de plus de deux millions d'appels.

Pièges :

- ▶ **Mémoïser un algorithme dont les sous-problèmes ne se recoupent pas.** Le tri fusion ne rencontre jamais deux fois la même sous-liste : le dictionnaire ne ferait qu'occuper de la mémoire. Vérifie le recoupement avant de transformer.
- ▶ **Mettre le dictionnaire par défaut dans la signature.** `memo={}` est créé une seule fois, à la définition, et **partagé** entre tous les appels : deux calculs indépendants se contaminent. Toujours `memo=None`.
- ▶ **Oublier d'enregistrer avant de renvoyer.** La fonction reste correcte et reste lente : c'est le pire cas, parce que rien ne le signale.
- ▶ **Choisir une clé incomplète.** Si deux arguments influencent le résultat et que la clé n'en retient qu'un, la mémoïsation renvoie la valeur d'un autre sous-problème : l'algorithme devient faux, pas seulement lent.

Vérifier : compare naïve et mémoïsée sur toutes les valeurs jusqu'à 20 — elles doivent coïncider exactement — puis compte les entrées du dictionnaire : il doit y en avoir autant que de sous-problèmes distincts, pas davantage.

S'entraîner : exercices C12 du chapitre.

Exercice 1

Écrire une fonction récursive `compter_feuilles(arbre)` qui renvoie le nombre de **feuilles** (nœuds sans enfant) d'un arbre binaire représenté avec la classe `Noeud` du cours. Tester sur l'arbre `Noeud(1, Noeud(2, Noeud(4), Noeud(5)), Noeud(3))`, qui possède 3 feuilles.

Exercice 2

En utilisant les fonctions `insérer` et `rechercher` du cours, construire un arbre binaire de recherche à partir des valeurs 7, 3, 9, 1, 5 insérées dans cet ordre. Vérifier ensuite que 5 est présent dans l'arbre et que 4 n'y est pas.

Exercice 3

On modélise un petit réseau de métro par le graphe (non orienté) suivant, où chaque station est reliée à ses stations directement accessibles :

```
1 metro = {
2     "A": ["B"],
3     "B": ["A", "C", "D"],
4     "C": ["B", "E"],
5     "D": ["B", "E"],
6     "E": ["C", "D"],
7 }
```

En utilisant la fonction `chemin` du cours (recherche de chemin par BFS), déterminer le trajet trouvé de "A" à "E". Combien de stations ce trajet compte-t-il (départ et arrivée compris) ?

Exercice 4

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. Une version récursive naïve de son calcul recalcule un très grand nombre de fois les mêmes valeurs.

1. Écrire une fonction `fib_memo(n, memo=None)` qui calcule F_n par **programmation dynamique** (mémoïsation), en s'inspirant de `rendu_memo` du cours.
2. Vérifier que $F_0, F_1, \dots, F_9$ vaut 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, et que $F_{30} = 832\,040$.

Exercice 5 ♦♦

Écrire, selon le principe **diviser pour régner** (comme le tri fusion du cours), une fonction récursive `maximum_dpr(liste)` qui renvoie le plus grand élément d'une liste non vide : diviser la liste en deux moitiés, trouver récursivement le maximum de chaque moitié, puis combiner en renvoyant le plus grand des deux. Quel est le cas de base ?

ALGORITHMES SUR LES ARBRES ET LES GRAPHEs — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] La taille d'un arbre binaire se calcule récursivement par :
 - A. $1 + \text{taille}(\text{gauche}) + \text{taille}(\text{droite})$, l'arbre vide ayant pour taille 0.
 - B. $1 + \max(\text{taille}(\text{gauche}), \text{taille}(\text{droite}))$.
 - C. le nombre de feuilles seulement.
 - D. $\text{taille}(\text{gauche}) \times \text{taille}(\text{droite})$.

Capacité C2

2. [Q2] Selon la convention du cours, la hauteur d'un arbre réduit à un seul nœud vaut :
 - A. 0.
 - B. -1.
 - C. 1.
 - D. indéfinie.

Capacité C3

3. [Q3] Le parcours INFIXE d'un arbre binaire visite :
 - A. la racine, puis le sous-arbre gauche, puis le sous-arbre droit.
 - B. le sous-arbre gauche, puis la racine, puis le sous-arbre droit.
 - C. le sous-arbre gauche, puis le sous-arbre droit, puis la racine.
 - D. les nœuds niveau par niveau, de haut en bas.

Capacité C4

4. [Q4] Le parcours en largeur d'un arbre s'appuie sur :
 - A. une pile.
 - B. un dictionnaire, et rien d'autre.
 - C. une file.
 - D. aucune structure auxiliaire.

Capacité C5

5. [Q5] Dans un arbre binaire de recherche, la recherche d'une clé plus petite que celle de la racine se poursuit :
 - A. en repartant de la racine.
 - B. dans les deux sous-arbres, pour être sûr.
 - C. dans le sous-arbre droit uniquement.
 - D. dans le sous-arbre gauche uniquement.

Capacité C6

6. [Q6] Insérer successivement les clés 1, 2, 3, 4 dans un arbre binaire de recherche vide produit :
 - A. un arbre dégénéré, de hauteur 3, semblable à une liste chaînée.
 - B. un arbre parfaitement équilibré de hauteur 1.

- C. un arbre où 4 devient la racine.
- D. une erreur, les clés devant être insérées dans le désordre.

Capacité C7

- 7. [Q7] Le parcours en largeur d'un graphe, à partir d'un sommet de départ, visite les sommets :
 - A. dans l'ordre croissant de leur numéro.
 - B. par distance croissante au sommet de départ, comptée en nombre d'arêtes.
 - C. en descendant d'abord le plus loin possible le long d'un chemin.
 - D. dans un ordre qui dépend du hasard.

Capacité C8

- 8. [Q8] Le parcours en profondeur d'un graphe s'implémente naturellement :
 - A. après un tri préalable des sommets.
 - B. avec une file, et elle seule.
 - C. par récursion, ou au moyen d'une pile explicite.
 - D. par une recherche dichotomique.

Capacité C9

- 9. [Q9] Lors du parcours d'un graphe non orienté, on détecte un cycle quand on rencontre un sommet déjà visité qui :
 - A. est le sommet de départ.
 - B. possède le plus grand nombre de voisins.
 - C. n'a aucun voisin.
 - D. n'est pas le parent direct dans le parcours.

Capacité C10

- 10. [Q10] Pour reconstituer le chemin trouvé entre deux sommets par un parcours, il faut :
 - A. mémoriser, pour chaque sommet visité, celui depuis lequel on l'a atteint.
 - B. relancer le parcours depuis l'arrivée.
 - C. trier les sommets par distance.
 - D. conserver uniquement la liste des sommets visités, dans l'ordre.

Capacité C11

- 11. [Q11] La méthode « diviser pour régner » consiste à :
 - A. traiter les données une par une, du début à la fin.
 - B. découper le problème en sous-problèmes de même nature, les résoudre, puis combiner leurs solutions.
 - C. essayer toutes les solutions possibles avant de choisir.
 - D. mémoriser les résultats déjà calculés pour ne pas les recalculer.

Capacité C12

- 12. [Q12] La programmation dynamique est particulièrement utile lorsque :
 - A. les sous-problèmes ne se recoupent jamais.
 - B. le problème n'admet aucune formulation récursive.
 - C. les mêmes sous-problèmes réapparaissent plusieurs fois.
 - D. il n'existe qu'un seul sous-problème.

Capacité C13

- 13. [Q13] À la différence de la recherche naïve, l'algorithme de Boyer-Moore compare le motif au texte :
 - A. dans un ordre tiré au hasard.
 - B. en ignorant complètement le texte.
 - C. sur le seul premier caractère.
 - D. de droite à gauche, en s'appuyant sur un prétraitement du motif.

2 Architectures matérielles, systèmes d'exploitation et réseaux

2.1 Le système sur puce

DÉFINITION 1

Un **système sur puce** (*System on Chip*, SoC) intègre sur un unique circuit intégré plusieurs composants qui, historiquement, occupaient des circuits séparés : un ou plusieurs **processeurs**, de la **mémoire**, des **interfaces** de communication (Wi-Fi, Bluetooth, USB...), et parfois des **accélérateurs** spécialisés (traitement graphique, calcul pour l'intelligence artificielle, traitement du signal audio).

◆ **PROPRIÉTÉ Avantages de l'intégration** Intégrer plusieurs composants sur une même puce présente trois avantages principaux :

- ▶ **compacité** : un seul circuit remplace plusieurs cartes séparées, ce qui réduit la taille du produit final ;
- ▶ **consommation énergétique réduite** : les distances entre composants, très courtes sur une même puce, réduisent les pertes électriques liées à la communication entre eux ;
- ▶ **coût de fabrication** : produire une seule puce en grande série revient souvent moins cher qu'assembler plusieurs composants distincts.

⚠ ERREUR FRÉQUENTE

Croire que l'intégration n'a que des avantages. L'inconvénient principal d'un système sur puce est la perte de **modularité** : si un seul composant est défaillant ou devient obsolète (par exemple la mémoire), c'est toute la puce qu'il faut remplacer, alors que des composants séparés auraient pu être changés individuellement.

✦ POUR ALLER PLUS LOIN

Certains systèmes sur puce embarquent également des **FPGA** (circuits reconfigurables) pour adapter certains traitements sans changer de puce physique, ou des zones mémoire dédiées à la sécurité (*secure enclave*) isolées du reste du système.

2.2 Processus : création, ordonnancement, interblocage

DÉFINITION 1

Un **processus** est un programme en cours d'exécution, avec son propre espace mémoire et son propre état (registres, pile d'appels). La **création** d'un processus consiste, pour le système d'exploitation, à lui allouer de la mémoire, à charger le code du programme, et à l'inscrire dans la liste des processus à exécuter.

DÉFINITION 2

Un seul processeur ne peut exécuter qu'une instruction à la fois : quand plusieurs processus doivent s'exécuter « en même temps », l'**ordonnanceur** du système d'exploitation leur alloue le processeur à tour de rôle, pendant de courtes tranches de temps, donnant l'illusion d'une exécution simultanée.

Exemple. La politique **tourniquet** (*round-robin*) alloue à chaque processus une tranche de temps fixe, dans l'ordre, en boucle :

```
1 def tourniquet(processus, tranche, duree_totale):
2     ordre_execution = []
3     file = list(processus)
4     temps_restant = dict(duree_totale)
5     while file:
6         p = file.pop(0)
7         execute = min(tranche, temps_restant[p])
8         ordre_execution.append((p, execute))
9         temps_restant[p] -= execute
10        if temps_restant[p] > 0:
11            file.append(p)
12    return ordre_execution
```

⚠ ERREUR FRÉQUENTE

Confondre concurrence et parallélisme. Sur un processeur à un seul cœur, plusieurs processus s'exécutent en **concurrence** (à tour de rôle, jamais réellement simultanément) : c'est l'ordonnancement rapide qui donne l'illusion du parallélisme. Sur un processeur multi-cœurs, plusieurs processus peuvent en revanche s'exécuter en **parallélisme** réel, un par cœur.

DÉFINITION 3

Un **interblocage** (*deadlock*) survient lorsque plusieurs processus attendent chacun une ressource retenue par un autre processus du même groupe, formant un cycle d'attente dont aucun ne peut sortir seul.

Exemple. Deux processus, chacun retenant une ressource et attendant celle de l'autre :

```
1 # Processus P1 : retient R1, attend R2
2 # Processus P2 : retient R2, attend R1
3 attente = {"P1": "R2", "P2": "R1"}
4 retenue_par = {"R1": "P1", "R2": "P2"}
```

◆ **PROPRIÉTÉ Mise en évidence par un graphe d'attente** Un interblocage se traduit par un **cycle** dans le graphe orienté où chaque processus pointe vers la ressource qu'il attend, et chaque ressource pointe vers le processus qui la retient. On retrouve ainsi la détection de cycle vue dans le chapitre *Algorithmique*, appliquée cette fois à un graphe de dépendances entre processus et ressources.

2.3 Protocoles de routage

DÉFINITION 1

Un réseau se modélise par un **graphe** dont les sommets sont des routeurs et les arêtes des liaisons directes. **Router** un paquet consiste à choisir, à chaque routeur, la liaison à emprunter pour progresser vers la destination. Le **protocole de routage** détermine comment cette route est calculée.

◆ **PROPRIÉTÉ RIP : minimiser le nombre de sauts** Le protocole **RIP** (*Routing Information Protocol*) choisit la route qui minimise le **nombre de sauts** (de routeurs traversés), sans tenir compte de la capacité ou de la congestion des liaisons. C'est exactement ce que calcule un parcours en largeur (BFS) sur le graphe du réseau.

◆ **PROPRIÉTÉ OSPF : minimiser un coût** Le protocole **OSPF** (*Open Shortest Path First*) associe un **coût** à chaque liaison (reflétant par exemple sa bande passante) et choisit la route qui minimise la somme des coûts, pas le nombre de sauts. Une route avec plus de sauts mais de meilleure capacité peut ainsi être préférée à une route plus courte mais plus lente.

Exemple. Sur ce réseau, la route au moins de sauts (RIP) et la route au moindre coût (OSPF) diffèrent :

```
1 reseau_couts = {
2   "A": {"B": 1, "C": 5},
3   "B": {"A": 1, "D": 1},
4   "C": {"A": 5, "D": 1},
5   "D": {"B": 1, "C": 1},
6 }
```

De A vers D : la route A-C-D compte 2 sauts (comme A-B-D), mais coûte $5 + 1 = 6$, alors que A-B-D compte aussi 2 sauts pour un coût de $1 + 1 = 2$: ici les deux critères coïncident. En revanche, si l'on ajoute une liaison directe coûteuse A-D de coût 3, RIP choisirait ce trajet à **un seul saut**, alors qu'OSPF préférerait toujours A-B-D (coût $2 < 3$).

⚠ ERREUR FRÉQUENTE

Croire qu'un protocole de routage choisit toujours le chemin « le plus court » au sens intuitif. Ce que minimise le protocole dépend entièrement de sa **métrique** : RIP minimise le nombre de sauts (route A-D directe préférée dans l'exemple), OSPF minimise un coût cumulé (route A-B-D préférée si son coût total est plus faible), même si cette dernière traverse davantage de routeurs.

2.4 Chiffrement symétrique et asymétrique

DÉFINITION 1

Le **chiffrement symétrique** utilise une **unique clé secrète**, partagée à l'avance entre l'émetteur et le destinataire, pour chiffrer et déchiffrer un message. Il est rapide, mais suppose que les deux parties aient pu échanger la clé de façon sûre au préalable.

Exemple. Un chiffrement symétrique très simplifié par **XOR** avec une clé répétée (principe illustratif, non utilisé tel quel en pratique) :

```
1 def chiffrer_xor(message, cle):
2     return bytes(o ^ cle[i % len(cle)] for i, o in enumerate(message))
3
4 def déchiffrer_xor(chiffre, cle):
5     return chiffrer_xor(chiffre, cle) # XOR est sa propre inverse
```

DÉFINITION 2

Le **chiffrement asymétrique** utilise une **paire de clés** liées mathématiquement : une **clé publique**, diffusée librement, et une **clé privée**, gardée secrète. Ce qui est chiffré avec la clé publique ne peut être déchiffré qu'avec la clé privée correspondante. Il est plus lent que le chiffrement symétrique, mais ne nécessite **aucun** échange préalable de secret.

♦ **PROPRIÉTÉ Sécuriser HTTPS : le meilleur des deux mondes** Une connexion **HTTPS** combine les deux : au début de la connexion, le client et le serveur utilisent un protocole **asymétrique** pour échanger, en toute sécurité sur un réseau public, une **clé symétrique** temporaire (dite *clé de session*). Toute la suite de l'échange (les pages web, les données du formulaire...) est ensuite chiffrée avec cette clé symétrique, beaucoup plus rapide à utiliser pour de gros volumes de données.

⚠ ERREUR FRÉQUENTE

Croire que HTTPS chiffre tout avec le protocole asymétrique de bout en bout. Ce serait beaucoup trop lent pour des échanges volumineux : le chiffrement asymétrique sert uniquement à échanger la clé symétrique de session au tout début de la connexion, puis c'est le chiffrement symétrique, bien plus rapide, qui protège le reste des données échangées.

+ POUR ALLER PLUS LOIN

Le chiffrement asymétrique repose sur des problèmes mathématiques réputés difficiles à inverser sans la clé privée (par exemple, la factorisation de grands nombres entiers pour l'algorithme RSA). Ce fondement mathématique est étudié plus en détail dans le programme de mathématiques expertes (arithmétique).

④ MÉTHODE M1 Dérouler un ordonnancement de processus

Quand l'utiliser ? Quand l'énoncé donne des processus avec leurs durées, une politique d'ordonnancement et parfois un quantum, puis demande le chronogramme, l'ordre d'exécution, ou un temps d'attente. Le code du cours ne se déroule pas dans la tête : il faut un tableau.

Pas à pas :

1. **Relève les données** : chaque processus avec sa durée totale, sa date d'arrivée si l'énoncé en donne une, et le quantum s'il y en a un. Une date d'arrivée non précisée vaut 0.
2. **Trace un tableau** à quatre colonnes : instant, processus élu, durée consommée, temps restant de chacun. Une ligne par tranche, jamais par processus.
3. **Applique la politique pour élire**. Tourniquet : le premier de la file, et le processus qui vient de s'exécuter repart **à la fin** s'il lui reste du travail. Plus court d'abord : celui dont le temps restant est le plus petit parmi les arrivés.
4. **Consomme le minimum entre le quantum et le temps restant**. Un processus à qui il reste 1 ms avec un quantum de 3 ms ne consomme que 1 ms : il libère le processeur en avance.
5. **Remets à jour la file** avant la ligne suivante — les arrivées d'abord, le processus préempté ensuite. Cet ordre change le résultat.
6. **Recommence** jusqu'à ce que tous les temps restants soient nuls, puis réponds à la question posée : ordre d'exécution, instant de fin de chacun, ou temps d'attente.

Exemple : trois processus arrivés en même temps, tourniquet de quantum 2. P_1 dure 5, P_2 dure 3, P_3 dure 1.

File $[P_1, P_2, P_3]$. P_1 prend 2 (reste 3) et repart derrière : $[P_2, P_3, P_1]$. P_2 prend 2 (reste 1) : $[P_3, P_1, P_2]$. P_3 prend 1 et **termine** — il ne consomme pas les deux : $[P_1, P_2]$. P_1 prend 2 (reste 1) : $[P_2, P_1]$. P_2 prend son dernier 1 et termine. P_1 prend son dernier 1 et termine.

Chronogramme : $P_1(2), P_2(2), P_3(1), P_1(2), P_2(1), P_1(1)$. Total 9, ce qui est bien $5 + 3 + 1$: aucun temps n'est créé ni perdu.

P_3 , le plus court, termine à l'instant 5 ; P_1 , le plus long, à l'instant 9.

Pièges :

- **Faire consommer le quantum entier à un processus presque fini**. P_3 ne consomme que 1 ms, pas 2 : le total du chronogramme doit valoir exactement la somme des durées, ni plus ni moins.
- **Oublier de remettre le processus préempté à la fin de la file**. Le tourniquet devient un « premier arrivé premier servi », et le processus le plus court attend la fin de tous les autres.
- **Placer le processus préempté avant les nouvelles arrivées**. Quand l'énoncé donne des dates d'arrivée, l'ordre d'insertion change le chronogramme : les arrivants d'abord.
- **Croire qu'un grand quantum est toujours meilleur**. Avec un quantum supérieur à la plus longue durée, il n'y a plus de rotation du tout, et les processus courts attendent le maximum.

Vérifier : additionne les tranches de ton chronogramme — le total doit être exactement la somme des durées. Puis vérifie que chaque processus a reçu sa durée complète, ni plus ni moins. Ces deux contrôles attrapent presque toutes les erreurs de recopie.

S'entraîner : exercices C3 du chapitre.

④ MÉTHODE M2 Déterminer la route empruntée selon la métrique du protocole

Quand l'utiliser ? Quand l'énoncé donne un réseau de routeurs, éventuellement avec des coûts de liaison, et demande quelle route un paquet emprunte « avec RIP » ou « avec OSPF ». La question n'est pas « quel est le chemin le plus court » : c'est « que minimise ce protocole-là ».

Pas à pas :

1. **Écris la métrique du protocole, en toutes lettres.** RIP minimise le **nombre de sauts**, c'est-à-dire le nombre de liaisons traversées, sans regarder les coûts. OSPF minimise la **somme des coûts** des liaisons traversées, sans regarder le nombre de sauts.
2. **Énumère les routes possibles** du départ à l'arrivée, sans repasser deux fois par le même routeur. Sur les réseaux des exercices, elles sont peu nombreuses : liste-les toutes plutôt que d'en deviner une.
3. **Calcule la métrique de chaque route**, dans une colonne par protocole demandé. Pour RIP, compte les liaisons — pas les routeurs. Pour OSPF, additionne les coûts.
4. **Choisis le minimum**, protocole par protocole. Si deux routes sont à égalité, dis-le : l'énoncé attend alors le critère de départage qu'il aura précisé, ou la mention des deux routes.
5. **Compare les deux réponses.** Quand elles diffèrent, explique pourquoi : c'est exactement ce que la question cherche à faire dire.

Exemple : réseau A, B, C, D . Liaisons et coûts : $A-B$ de coût 1, $B-D$ de coût 1, $A-C$ de coût 5, $C-D$ de coût 1, et une liaison directe $A-D$ de coût 3. Quelle route de A vers D ?

Routes possibles : $A-D$ (1 saut, coût 3), $A-B-D$ (2 sauts, coût 2), $A-C-D$ (2 sauts, coût 6). RIP minimise les sauts : il choisit $A-D$, un seul saut, alors que cette liaison coûte 3.

OSPF minimise le coût : il choisit $A-B-D$, coût 2, bien qu'elle traverse un routeur de plus.

Les deux protocoles donnent des routes **différentes** sur le même réseau, et aucune n'est « fausse » : chacune est optimale pour sa métrique.

Pièges :

- ▶ **Chercher « le chemin le plus court » sans dire au sens de quoi.** La question n'a pas de réponse tant que la métrique n'est pas fixée : RIP et OSPF choisissent ici deux routes différentes sur le même réseau.
- ▶ **Compter les routeurs au lieu des liaisons.** $A-B-D$ traverse 3 routeurs mais fait 2 sauts. Le décalage d'une unité passe souvent inaperçu au classement et fausse la réponse chiffrée.
- ▶ **Croire qu'une route à moins de sauts coûte forcément moins cher.** C'est justement le contre-exemple de l'énoncé : la liaison directe est la plus courte en sauts et la plus chère en coût.
- ▶ **Généraliser la divergence.** Sans la liaison directe, les deux protocoles choisissent la même route. La divergence vient du réseau donné, pas d'une opposition de principe entre les protocoles.

Vérifier : contrôle que tu as bien listé **toutes** les routes sans répétition de routeur, puis que la route retenue a la plus petite valeur dans sa colonne — et seulement dans la sienne. Si les deux protocoles donnent la même route, dis-le : c'est une réponse, pas un échec.

S'entraîner : exercices C5 du chapitre.

Exercice 1

En utilisant la fonction tourniquet du cours, déterminer l'ordre d'exécution pour deux processus X (durée totale 4) et Y (durée totale 6), avec une tranche de temps de 3. Combien de tranches de temps sont-elles nécessaires au total pour que les deux processus se terminent ?

Exercice 2

Trois processus se partagent trois ressources : P_1 retient R_1 et attend R_2 ; P_2 retient R_2 et attend R_3 ; P_3 retient R_3 et attend R_1 .

1. Représenter cette situation par les deux dictionnaires attente et retenue du cours.
2. Y a-t-il un interblocage ? Justifier en décrivant le cycle d'attente.

Exercice 3

On donne le réseau de coûts suivant :

```

1 reseau = {
2     "A": {"B": 2, "C": 10},
3     "B": {"A": 2, "C": 2},
4     "C": {"A": 10, "B": 2},
5 }

```

1. Quelle route RIP (minimisant le nombre de sauts) choisirait-il de A vers C ? En utilisant bfs_sauts du cours, combien de sauts compte-t-elle ?
2. En utilisant plus_court_cout du cours, quel est le coût de la route effectivement la moins coûteuse de A vers C ? Ce n'est pas la route directe : pourquoi ?

Exercice 4

En utilisant chiffrer_xor du cours, chiffrer le message b"BAC2027" avec la clé b"NSI", puis vérifier qu'appliquer la même fonction une seconde fois sur le résultat, avec la même clé, redonne le message d'origine. Pourquoi ce chiffrement est-il qualifié de **symétrique** ?

ARCHITECTURES MATÉRIELLES, SYSTÈMES D'EXPLOITATION ET RÉSEAUX — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Par rapport à des composants séparés, le principal inconvénient d'un système sur puce est :
 - A. la perte de modularité : un composant défaillant ne peut pas être remplacé seul.
 - B. une consommation d'énergie plus élevée.
 - C. un encombrement plus important.
 - D. un coût de fabrication systématiquement supérieur.

Capacité C2

2. [Q2] Créer un processus consiste, pour le système d'exploitation, à :
 - A. compiler le code source du programme.
 - B. lui allouer de la mémoire et l'inscrire parmi les processus à exécuter.
 - C. arrêter tous les autres processus.
 - D. lui attribuer une adresse IP.

Capacité C3

3. [Q3] L'ordonnanceur du système d'exploitation a pour rôle de :
 - A. trier les fichiers sur le disque.
 - B. compiler les programmes avant leur lancement.
 - C. décider quel processus prêt obtient le processeur, et pour combien de temps.
 - D. supprimer les processus terminés de la mémoire, et rien d'autre.

Capacité C4

4. [Q4] Dans le graphe d'attente des processus, un interblocage se manifeste par :
 - A. un cycle.
 - B. un arbre.
 - C. un graphe sans aucune arête.
 - D. un sommet isolé.

Capacité C5

5. [Q5] Le protocole de routage RIP choisit la route qui minimise :
 - A. le coût cumulé des liaisons traversées.
 - B. le nombre de sauts.
 - C. la latence mesurée en temps réel.

D. le nombre d'utilisateurs connectés.

Capacité C6

6. [Q6] Le chiffrement asymétrique repose sur :

- A. une unique clé secrète, partagée par les deux correspondants.
- B. l'absence de toute clé.
- C. une paire de clés, l'une publique et l'autre privée.
- D. une clé différente pour chaque caractère du message.

Capacité C7

7. [Q7] Au cours d'une connexion HTTPS, le chiffrement asymétrique sert principalement à :

- A. chiffrer la totalité des données échangées pendant toute la session.
- B. compresser les données avant leur envoi.
- C. afficher le cadenas dans le navigateur.
- D. convenir en sécurité d'une clé symétrique de session.

3 Bases de données relationnelles

3.1 Le modèle relationnel

DÉFINITION 1

Le **modèle relationnel** organise les données en **relations** (tables). Une relation est constituée de **tuples** (lignes), chacun décrivant une occurrence, et d'**attributs** (colonnes), chacun défini sur un **domaine** (l'ensemble des valeurs possibles, par exemple entier ou texte). Le **schéma** d'une relation liste le nom de la relation et le nom de chacun de ses attributs.

DÉFINITION 2

Une **clé primaire** est un attribut (ou groupe d'attributs) dont la valeur identifie de façon **unique** chaque tuple d'une relation. Une **clé étrangère** est un attribut d'une relation dont la valeur fait référence à la clé primaire d'une autre relation : c'est ainsi que deux relations sont mises en lien.

Exemple. Une bibliothèque gère trois relations liées entre elles :

```
1 Livre(id, titre, auteur, année)
2 Usager(id, nom, prenom)
3 Emprunt(id, id_livre, id_usager, date_emprunt, date_retour)
```

Dans Emprunt, `id_livre` est une clé étrangère vers `Livre.id`, et `id_usager` une clé étrangère vers `Usager.id` : chaque emprunt référence un livre et un usager précis, sans dupliquer leurs informations.

◆ **PROPRIÉTÉ Structure et contenu** La **structure** d'une base de données (son schéma : relations, attributs, clés, contraintes) est stable dans le temps. Le **contenu** (les tuples effectivement stockés) évolue à chaque insertion, modification ou suppression. Modifier le contenu ne change jamais la structure ; modifier la structure (ajouter une colonne, par exemple) laisse le contenu existant intact autant que possible.

⚠ ERREUR FRÉQUENTE

Confondre relation et fichier plat. Stocker toutes les informations d'une bibliothèque dans une seule table `Emprunt(id, titre_livre, auteur, nom_usager, prenom_usager, ...)` **duplique** le titre et l'auteur à chaque emprunt du même livre : c'est une **anomalie de schéma**. Une modification (correction d'une faute dans un titre) doit alors être répercutée sur toutes les lignes concernées, au risque d'incohérences. Séparer `Livre`, `Usager` et `Emprunt` en trois relations liées par des clés étrangères élimine cette duplication.

+ POUR ALLER PLUS LOIN

Ce principe de séparation pour éviter les anomalies porte un nom en théorie des bases de données : la **normalisation** (formes normales 1NF, 2NF, 3NF). Le programme de Terminale ne l'exige pas formellement, mais en comprendre l'intuition (une information ne doit être stockée qu'à un seul endroit) aide à concevoir des schémas robustes.

3.2 Le système de gestion de bases de données

DÉFINITION 1

Un **système de gestion de bases de données** (SGBD) est un logiciel qui prend en charge le stockage, l'interrogation et la modification des données d'une base, en offrant quatre services principaux :

- ▶ la **persistance** : les données restent stockées durablement, indépendamment de l'exécution d'un programme particulier ;
- ▶ la **concurrence** : plusieurs utilisateurs ou programmes peuvent accéder à la base simultanément sans corrompre les données ;
- ▶ l'**efficacité** : les requêtes sur de grands volumes de données restent rapides, notamment grâce à des index ;
- ▶ la **sécurisation** : l'accès aux données est contrôlé (droits d'utilisateurs) et leur intégrité est préservée (contraintes, transactions).

Exemple. La persistance : une base créée dans un fichier reste accessible après la fermeture du programme qui l'a créée.

```
1 import sqlite3
2
3 conn = sqlite3.connect("bibliotheque.db")
4 conn.execute("CREATE TABLE IF NOT EXISTS Livre(id INTEGER PRIMARY KEY, titre TEXT)")
5 conn.execute("INSERT INTO Livre(titre) VALUES ('1984')")
6 conn.commit()
7 conn.close()
8 # Le fichier bibliotheque.db contient désormais la table et sa ligne,
9 # meme apres la fin du programme.
```

◆ **PROPRIÉTÉ Transaction** Une **transaction** regroupe plusieurs opérations en un tout indivisible : soit toutes les opérations réussissent et sont **validées** (COMMIT), soit une échoue et toutes sont **annulées** (ROLLBACK), pour ne jamais laisser la base dans un état intermédiaire incohérent.

⚠ ERREUR FRÉQUENTE

Confondre un SGBD et un simple fichier. Un fichier texte ou CSV stocke des données mais n'offre **aucun** des quatre services ci-dessus : pas de contrôle de concurrence, pas de garantie d'intégrité, pas de langage d'interrogation efficace sur de gros volumes. C'est précisément pour ces raisons qu'on utilise un SGBD dès que les données doivent être partagées, protégées ou interrogées de façon complexe.

3.3 Interroger une base : SELECT, FROM, WHERE

DÉFINITION 1

Le langage **SQL** (*Structured Query Language*) permet d'interroger et de manipuler une base relationnelle. Une requête d'interrogation combine des **clauses**, chacune introduite par un **mot clé** : **SELECT** (quels attributs afficher), **FROM** (dans quelle relation), et optionnellement **WHERE** (quelle condition sur les tuples).

Exemple. Base d'exemple filée sur tout le chapitre (bibliothèque) :

```
1 CREATE TABLE Livre(id INTEGER PRIMARY KEY, titre TEXT, auteur TEXT, année INTEGER);
2 INSERT INTO Livre VALUES
3 (1, 'Le Petit Prince', 'Saint-Exupéry', 1943),
4 (2, '1984', 'Orwell', 1949),
5 (3, 'La Peste', 'Camus', 1947);
```

Afficher uniquement les titres et auteurs de tous les livres :

```
1 SELECT titre, auteur FROM Livre;
```

◆ **PROPRIÉTÉ Filtrer avec WHERE** La clause **WHERE** ne conserve que les tuples vérifiant une **condition**. Les opérateurs de comparaison usuels (=, <, >, <=, >=, <> pour différent) se combinent avec **AND**, **OR**, **NOT** pour des conditions composées ; **LIKE** permet une recherche approchée sur du texte (motif avec le joker %).

Exemple. Livres parus après 1945 :

```
1 SELECT titre FROM Livre WHERE année > 1945;
```

Livres d'Orwell ou de Camus :

```
1 SELECT titre FROM Livre WHERE auteur = 'Orwell' OR auteur = 'Camus';
```

⚠ ERREUR FRÉQUENTE

Oublier les guillemets autour d'une chaîne de caractères. **WHERE** auteur = Orwell est une erreur : SQL interprète Orwell sans guillemets comme un nom de colonne inexistant. Il faut écrire **WHERE** auteur = 'Orwell', avec des apostrophes simples autour du texte.

3.4 Croiser les données : JOIN et ORDER BY

DÉFINITION 1

La clause **JOIN** combine des tuples de deux relations dont les valeurs coïncident sur une colonne commune, en général une clé étrangère et la clé primaire qu'elle référence. La syntaxe **JOIN ... ON ...** précise la condition de rapprochement.

Exemple. Reprenons Livre et une table Emprunt qui référence Livre.id :

```
1 CREATE TABLE Emprunt(id INTEGER PRIMARY KEY, id_livre INTEGER, date_emprunt TEXT);
```

```
2 INSERT INTO Emprunt VALUES (1, 2, '2026-02-01'), (2, 3, '2026-02-03');
3
4 SELECT Emprunt.date_emprunt, Livre.titre
5 FROM Emprunt
6 JOIN Livre ON Emprunt.id_livre = Livre.id;
```

Chaque ligne du résultat associe une date d'emprunt au **titre** du livre correspondant, sans jamais avoir stocké ce titre dans la table Emprunt.

◆ **PROPRIÉTÉ Trier avec ORDER BY** La clause ORDER BY trie les tuples du résultat selon un ou plusieurs attributs, par ordre croissant (ASC, par défaut) ou décroissant (DESC).

Exemple. Livres triés du plus récent au plus ancien :

```
1 SELECT titre, année FROM Livre ORDER BY année DESC;
```

⚠ ERREUR FRÉQUENTE

Oublier que ORDER BY se place après WHERE. L'ordre des clauses est fixe : SELECT ... FROM ... WHERE ... ORDER BY Écrire ORDER BY avant WHERE est une erreur de syntaxe.

3.5 Modifier une base : INSERT, UPDATE, DELETE

DÉFINITION 1

La clause **INSERT INTO** ajoute un ou plusieurs tuples à une relation. On précise la relation, éventuellement la liste des attributs renseignés, puis les valeurs avec VALUES.

Exemple.

```
1 INSERT INTO Livre(titre, auteur, année)
2 VALUES ('Fahrenheit 451', 'Bradbury', 1953);
```

◆ **PROPRIÉTÉ Mettre à jour avec UPDATE** La clause UPDATE ... SET ... WHERE ... modifie la valeur d'un ou plusieurs attributs pour les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont modifiés.**

Exemple. Corriger l'année de parution d'un livre :

```
1 UPDATE Livre SET année = 1954 WHERE titre = 'Fahrenheit 451';
```

DÉFINITION 2

La clause **DELETE FROM ... WHERE ...** supprime les tuples vérifiant une condition. **Sans clause WHERE, tous les tuples de la relation sont supprimés** (la table reste, mais devient vide).

Exemple. Supprimer les livres parus avant 1950 :

```
1 DELETE FROM Livre WHERE année < 1950;
```

⚠ ERREUR FRÉQUENTE

Exécuter un UPDATE ou un DELETE sans WHERE. C'est l'erreur la plus dangereuse en SQL : `DELETE FROM Livre;` (sans condition) supprime **tous** les livres, et `UPDATE Livre SET annee = 2000;` écrase l'année de **tous** les livres. Il faut toujours vérifier, avant d'exécuter une modification, quels tuples seraient concernés en testant d'abord la condition avec un `SELECT`.

» **Python À la machine.** Avant tout `UPDATE` ou `DELETE`, exécute d'abord le `SELECT` correspondant avec la même clause `WHERE` pour vérifier quels tuples seront affectés.

🔍 MÉTHODE M1 Repérer les anomalies d'un schéma relationnel

Quand l'utiliser ? Quand l'énoncé donne un schéma — une ou plusieurs relations avec leurs attributs — et demande de « repérer les anomalies » ou de « critiquer la conception ». Ce n'est pas une question d'opinion : chaque anomalie se montre en exhibant une opération qui tourne mal.

Pas à pas :

1. **Cherche l'information stockée deux fois.** Passe chaque attribut en revue et demande-toi : cette valeur peut-elle apparaître à l'identique sur plusieurs lignes ? Le titre d'un livre répété à chaque emprunt est le cas d'école.
2. **Nomme l'anomalie que la redondance provoque**, en trois temps. *Mise à jour* : corriger une faute dans le titre oblige à modifier toutes les lignes, et en oublier une rend la base incohérente. *Insertion* : on ne peut pas enregistrer un livre tant que personne ne l'a emprunté. *Suppression* : supprimer le dernier emprunt efface aussi le livre.
3. **Vérifie que chaque relation a une clé primaire**, et qu'elle identifie bien **un seul** tuple. Une relation sans clé primaire autorise les doublons parfaits, qu'aucune requête ne peut ensuite distinguer.
4. **Vérifie que chaque référence est une clé étrangère** pointant vers une clé primaire existante. Un attribut qui contient un numéro « par convention », sans contrainte déclarée, laisse entrer des références vers rien.
5. **Traque les attributs à valeurs multiples** : un attribut auteurs qui contiendrait « Orwell, Huxley » n'est pas interrogeable — aucune requête ne peut chercher proprement un auteur là-dedans.
6. **Propose la correction** : sortir l'information répétée dans sa propre relation, lui donner une clé primaire, et la référencer par une clé étrangère.

Exemple : `Emprunt(id, titre_livre, auteur, nom_usager, date)`.

Redondance : les colonnes `titre_livre` et `auteur` se répètent à chaque emprunt du même livre, et la colonne `nom_usager` se répète à chaque emprunt du même lecteur. La même information est donc stockée autant de fois qu'il y a d'emprunts.

Mise à jour : corriger « Orwel » en « Orwell » demande de modifier toutes les lignes de ce livre. En oublier une laisse deux orthographes dans la base, et une requête sur l'auteur en manquera une partie.

Insertion : on ne peut pas enregistrer un livre acquis mais jamais emprunté — il faudrait inventer un emprunt.

Suppression : supprimer le seul emprunt d'un livre efface toute trace de ce livre.

Correction : trois relations liées par deux clés étrangères.

- ▶ `Livre(id, titre, auteur)`
- ▶ `Usager(id, nom)`
- ▶ `Emprunt(id, id_livre, id_usager, date)`

Pièges :

- ▶ **Confondre redondance et répétition légitime.** Une date qui revient parce que deux emprunts ont eu lieu le même jour n'est pas une anomalie : la valeur se répète, mais elle décrit deux faits différents. L'anomalie, c'est de répéter la description d'un **même** objet.
- ▶ **S'arrêter au constat de redondance.** Dire « c'est redondant » ne vaut pas réponse : il faut exhiber l'opération qui tourne mal — mise à jour partielle, insertion impossible, suppression collatérale.
- ▶ **Oublier de vérifier les clés.** Une relation sans clé primaire et un attribut de référence sans contrainte sont deux anomalies distinctes de la redondance.
- ▶ **Proposer une correction qui déplace le problème.** Sortir le titre dans une relation Titre sans clé primaire ne corrige rien : la nouvelle relation hérite du même défaut.

Vérifier : pour chaque anomalie annoncée, écris la requête ou l'opération qui la manifeste — un `SELECT DISTINCT` qui renvoie deux orthographes du même auteur est une preuve, pas une impression. Puis vérifie que ta correction fait passer les trois opérations que l'ancien schéma ratait.

S'entraîner : exercices C3 du chapitre.

📌 MÉTHODE M2 Traduire une question en requête SQL

Quand l'utiliser ? Dès qu'un énoncé pose une question en français — « quels sont les titres des livres empruntés par Ada, du plus récent au plus ancien ? » — et attend une requête. Le cours donne la syntaxe de chaque clause ; ce qui se travaille ici, c'est l'ordre dans lequel on décide.

Pas à pas — décide dans cet ordre, écris dans l'ordre de SQL :

1. **Quelles colonnes veut-on VOIR ?** Souligne-les dans la question. Elles iront après `SELECT`. « Les titres » : une seule colonne, titre.
2. **Dans quelles relations vivent-elles ?** Si toutes les colonnes — celles affichées **et** celles qui servent aux conditions — sont dans une seule relation, tu n'as pas besoin de `JOIN`. Sinon, liste les relations nécessaires.
3. **Comment se raccordent-elles ?** Cherche le couple clé étrangère / clé primaire qui les relie, et écris-le tel quel : `JOIN Livre ON Emprunt.id_livre = Livre.id`. Une jointure sans condition produit **toutes** les combinaisons, ce qui n'est presque jamais ce qu'on veut.
4. **Quelle condition filtre les lignes ?** Traduis chaque contrainte de la question en comparaison, et relie-les par `AND` ou `OR`. Les chaînes de caractères prennent des apostrophes simples.
5. **Y a-t-il un ordre demandé ?** « du plus récent au plus ancien » donne `ORDER BY date DESC`. Sans consigne, ne trie pas.
6. **Écris la requête dans l'ordre de SQL** — `SELECT`, `FROM`, `JOIN`, `WHERE`, `ORDER BY` — puis relis-la à côté de la question, membre par membre.

Exemple : « les titres des livres empruntés par Ada, du plus récent au plus ancien ».

Colonnes vues : titre. *Relations :* titre est dans Livre, mais « empruntés par Ada » exige Emprunt et Usager : trois relations. *Raccords :* `Emprunt.id_livre = Livre.id` et `Emprunt.id_usager = Usager.id`. *Condition :* `Usager.nom = 'Ada'`. *Ordre :* `Emprunt.date DESC`.

```
SELECT Livre.titre
FROM Emprunt
JOIN Livre ON Emprunt.id_livre = Livre.id
JOIN Usager ON Emprunt.id_usager = Usager.id
WHERE Usager.nom = 'Ada'
ORDER BY Emprunt.date DESC;
```

Pièges :

- **Joindre sur la mauvaise égalité de clés.** `ON Emprunt.id = Livre.id` au lieu de `ON Emprunt.id_livre = Livre.id` donne un résultat **non vide** et faux : rien ne signale l'erreur, il faut relire la condition.
- **Oublier la condition de jointure.** `FROM Emprunt, Livre` sans `ON` produit le produit de toutes les lignes — ici 9 au lieu de 3.
- **Traduire « ou » par AND.** « Les livres de 1949 ou de 1953 » s'écrit avec `OR` ; avec `AND`, aucune ligne ne peut satisfaire les deux, et le résultat est vide.
- **Oublier les apostrophes autour d'un texte.** `WHERE nom = Ada` fait chercher à SQL une colonne appelée Ada, et la requête échoue.
- **Trier sans qu'on l'ait demandé, ou l'inverse.** `ORDER BY` se place après `WHERE`, et « du plus récent au plus ancien » impose `DESC`.

Vérifier : relis ta requête à côté de la question, morceau par morceau — chaque membre du `SELECT` correspond à un mot de la question, chaque `JOIN` à une relation traversée, chaque `WHERE` à une contrainte. Puis compte les lignes attendues sur les données de l'énoncé : un résultat dix fois trop grand est presque toujours une jointure sans condition.

S'entraîner : exercices C6 à C9 du chapitre.

Exercice 1 ♦

Un club de sport stocke tous ses adhérents dans une unique table :

```
1 Inscription(id, nom_adherent, prenom_adherent, nom_sport, jour_seance, nom_coach)
```

1. Un adhérent pratiquant deux sports différents apparaît-il une ou deux fois dans cette table ? Quel problème cela pose-t-il si l'on corrige une faute de frappe dans son nom ?
2. Proposer un découpage de cette table en plusieurs relations (avec leurs attributs) qui élimine ce problème. Préciser au moins une clé étrangère.

Exercice 2 ♦

On dispose de la table suivante :

```
1 CREATE TABLE Film(id INTEGER PRIMARY KEY, titre TEXT, realisateur TEXT, duree INTEGER);
2 INSERT INTO Film VALUES
3 (1, 'Le Voyage de Chihiro', 'Miyazaki', 125),
4 (2, 'Princesse Mononoke', 'Miyazaki', 134),
5 (3, 'Vice-versa', 'Docter', 95);
```

Écrire les requêtes SQL permettant d'afficher :

1. les titres des films réalisés par Miyazaki ;
2. les titres des films dont la durée dépasse 100 minutes.

Exercice 3 ♦♦

On dispose de deux tables :

```
1 Eleve(id, nom)
2 Note(id, id_eleve, matiere, valeur)
```

où `Note.id_eleve` est une clé étrangère vers `Eleve.id`. Écrire une requête SQL affichant, pour chaque note, le nom de l'élève, la matière et la valeur de la note, triées par valeur **décroissante**.

Exercice 4 ♦♦

On dispose de la table :

```
1 CREATE TABLE Stock(id INTEGER PRIMARY KEY, produit TEXT, quantité INTEGER);
2 INSERT INTO Stock VALUES (1, 'Cahier', 40), (2, 'Stylo', 0), (3, 'Règle', 12);
```

Écrire les requêtes SQL permettant, dans l'ordre, de :

1. ajouter un produit 'Gomme' avec une quantité de 25 ;
2. diminuer de 5 la quantité de 'Cahier' (sans réécrire la valeur numérique finale) ;
3. supprimer tous les produits dont la quantité en stock est nulle.

BASES DE DONNÉES RELATIONNELLES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Dans une relation, un attribut est défini sur :
 - A. un domaine, c'est-à-dire l'ensemble des valeurs qu'il peut prendre.
 - B. un tuple.
 - C. une clé étrangère, et rien d'autre.
 - D. une autre relation.

Capacité C2

2. [Q2] Le schéma d'une base de données décrit :
 - A. les valeurs actuellement enregistrées dans les tables.
 - B. la structure : les relations, leurs attributs, leurs domaines et leurs clés.
 - C. le nombre de lignes de chaque table.
 - D. l'historique des requêtes exécutées.

Capacité C3

3. [Q3] Répéter le titre et l'auteur d'un livre sur chaque ligne d'une table d'emprunts, au lieu de les placer dans une table dédiée, entraîne :
 - A. un gain de temps de calcul systématique.
 - B. une amélioration de la sécurité.
 - C. une duplication des données et un risque d'incohérence lors des mises à jour.
 - D. aucune conséquence particulière.

Capacité C4

4. [Q4] Le service de persistance d'un SGBD garantit que :
 - A. les mots de passe sont chiffrés.
 - B. les requêtes s'exécutent instantanément.
 - C. deux utilisateurs n'accèdent jamais à la base en même temps.
 - D. les données restent conservées durablement, même après l'arrêt du programme.

Capacité C5

5. [Q5] Dans la requête `SELECT nom FROM Client WHERE ville = 'Lyon';`, la clause `FROM` indique :
 - A. la relation dans laquelle les données sont puisées.
 - B. les colonnes à afficher.
 - C. la condition que les lignes doivent vérifier.
 - D. l'ordre d'affichage du résultat.

Capacité C6

6. [Q6] La requête `SELECT nom FROM Client WHERE ville = 'Lyon';` affiche :
 - A. toutes les colonnes des clients lyonnais.

- B. le seul nom des clients habitant Lyon.
- C. le nombre de clients lyonnais.
- D. toutes les villes des clients.

Capacité C7

7. [Q7] Écrire `WHERE ville = Lyon` sans apostrophes, au lieu de `WHERE ville = 'Lyon'` :
- A. ne change rien au résultat.
 - B. rend la requête plus rapide.
 - C. est une erreur : Lyon est alors compris comme un nom de colonne.
 - D. est obligatoire pour les textes courts.

Capacité C8

8. [Q8] La clause `JOIN B ON A.x = B.y` permet :
- A. de trier les résultats de la table A.
 - B. de renommer la colonne x en y.
 - C. de supprimer les doublons de A.
 - D. de rapprocher les tuples de A et de B dont les valeurs de x et de y coïncident.

Capacité C9

9. [Q9] Pour afficher les clients du plus jeune au plus âgé, on complète la requête par :
- A. `ORDER BY age ASC`
 - B. `WHERE age`
 - C. `GROUP BY age`
 - D. `ORDER BY age DESC`

Capacité C10

10. [Q10] Pour ajouter un client nommé Dupont, domicilié à Lyon, dans la table `Client(nom, ville)`, on écrit :
- A. `UPDATE Client SET nom = 'Dupont', ville = 'Lyon';`
 - B. `INSERT INTO Client (nom, ville) VALUES ('Dupont', 'Lyon');`
 - C. `SELECT 'Dupont', 'Lyon' FROM Client;`
 - D. `INSERT Client VALUES nom = 'Dupont';`

Capacité C11

11. [Q11] Dans une requête `UPDATE`, omettre la clause `WHERE` a pour effet :
- A. de faire échouer la requête.
 - B. de n'appliquer la modification qu'à la première ligne.
 - C. de modifier toutes les lignes de la table.
 - D. de créer une nouvelle table.

Capacité C12

12. [Q12] La requête `DELETE FROM Emprunt WHERE date_retour IS NOT NULL;` :
- A. supprime la table Emprunt tout entière.
 - B. vide la colonne `date_retour` des lignes concernées.
 - C. supprime toutes les lignes de la table, la condition étant ignorée.
 - D. supprime les lignes dont la date de retour est renseignée.



4 Histoire de l'informatique

4.1 Événements clés de l'histoire de l'informatique

◆ PROPRIÉTÉ Repères 1936–1948 : la machine universelle

- ▶ **1936** : Alan Turing formalise le concept de **machine universelle** (machine de Turing), capable en théorie d'exécuter tout algorithme calculable. C'est la première fois que les notions de **machine**, **algorithme**, **langage** et **information** sont pensées comme un tout cohérent.
- ▶ **1945** : John von Neumann décrit l'**architecture** qui porte son nom : programme et données stockés dans une même mémoire (voir chapitre Architectures matérielles).
- ▶ **1948** : construction des premiers ordinateurs électroniques à programme enregistré (par exemple le *Manchester Baby*).

◆ PROPRIÉTÉ De 1948 à aujourd'hui : croissance et diversification

- ▶ La puissance de calcul des ordinateurs a évolué de manière **exponentielle** (loi empirique de Moore, 1965 : doublement approximatif du nombre de transistors par circuit tous les deux ans).
- ▶ Les ordinateurs se sont **diversifiés** en taille, forme et usage : ordinateurs personnels, serveurs, fermes de calcul, téléphones, tablettes, montres connectées, objets connectés.
- ▶ **1969** : mise en service du réseau ARPANET, ancêtre d'**Internet**, qui relie aujourd'hui ordinateurs et objets connectés à l'échelle mondiale.

EXEMPLE Situer sur une frise chronologique : machine de Turing (1936), premiers ordinateurs à programme enregistré (1948), ARPANET (1969), micro-ordinateurs personnels (fin des années 1970), World Wide Web (1989-1991), smartphones (années 2000).

⚠ ERREUR FRÉQUENTE

Confondre Internet et le World Wide Web. Internet (réseau de réseaux, depuis 1969/ARPANET) est l'infrastructure de communication ; le Web (inventé par Tim Berners-Lee en 1989-1991) est un **service** qui fonctionne au-dessus d'Internet, au même titre que le courrier électronique. Le Web n'est qu'une des utilisations d'Internet.

4.2 Évolution des rôles du logiciel et du matériel

◆ PROPRIÉTÉ Des débuts au logiciel dominant

- ▶ Dans les tout premiers ordinateurs (fin des années 1940), le **matériel** était prédominant : programmer consistait souvent à reconfigurer physiquement des câblages ou à saisir du code machine directement.

- ▶ L'apparition des **langages de programmation** de haut niveau (Fortran 1957, puis de nombreux autres) et des **compilateurs** déplace progressivement la complexité vers le **logiciel** : le matériel devient plus générique et reconfigurable par programmation.
- ▶ Les **systèmes d'exploitation** (années 1960-1970) organisent le partage des ressources matérielles entre plusieurs programmes, ajoutant une couche logicielle essentielle entre l'utilisateur et la machine.

◆ **PROPRIÉTÉ Aujourd'hui : le logiciel omniprésent** Un même matériel (smartphone, ordinateur) exécute une multitude de logiciels différents grâce à des couches d'abstraction (système d'exploitation, machine virtuelle, navigateur). Le **coût de développement logiciel** dépasse souvent aujourd'hui le coût du matériel dans de nombreux projets numériques.

EXEMPLE Comparer un four à micro-ondes des années 1980 (circuits électroniques dédiés, comportement figé) à un four connecté moderne (matériel générique + logiciel embarqué mis à jour à distance) : le second illustre le basculement du rôle de spécialisation vers le logiciel.

⚠ ERREUR FRÉQUENTE

Croire que le matériel est devenu secondaire. Le matériel reste indispensable (sans lui, aucun logiciel ne peut s'exécuter) : c'est son **rôle** qui a évolué, passant de porteur de la logique spécifique à plateforme générique exécutant une logique portée par le logiciel.

Exercice 1 ◆

Remettre dans l'ordre chronologique les événements suivants, en précisant l'année de chacun :

- ▶ Mise en service du réseau ARPANET.
- ▶ Formalisation de la machine universelle par Alan Turing.
- ▶ Construction des premiers ordinateurs à programme enregistré.

Exercice 2 ◆◆

Un smartphone moderne peut exécuter des milliers d'applications différentes sans que son matériel change.

1. Expliquer, en une phrase, pourquoi cela illustre l'évolution du rôle du matériel vers plus de générique.
2. Citer deux couches logicielles (parmi système d'exploitation, machine virtuelle, navigateur, compilateur) qui permettent cette flexibilité, et expliquer brièvement le rôle de chacune.

HISTOIRE DE L'INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Alan Turing formalise le concept de machine universelle en :
 - A. 1936.
 - B. 1948.
 - C. 1969.
 - D. 1991.

2. [Q2] ARPANET, ancêtre d'Internet, est mis en service en :

- A. 1948.
- B. 1969.
- C. 1989.
- D. 2000.

Capacité C2

3. [Q3] Depuis les débuts de l'informatique, le rôle du matériel a évolué vers :

- A. une spécialisation croissante, chaque machine ne traitant qu'une seule tâche.
- B. une disparition progressive au profit du seul logiciel.
- C. davantage de généricité, la logique particulière étant portée par le logiciel.
- D. aucune évolution notable.

4. [Q4] Internet et le World Wide Web sont dans le rapport suivant :

- A. ce sont deux noms de la même chose.
- B. ce sont deux réseaux physiques distincts et indépendants.
- C. Internet est un service qui fonctionne au-dessus du Web.
- D. le Web est un service qui fonctionne au-dessus du réseau Internet.



5 Langages, récursivité et paradigmes de programmation

5.1 Programme, donnée et calculabilité

DÉFINITION 1

Un programme est lui-même une suite de caractères (son code source) ou d'octets (son code compilé) : c'est donc une **donnée** comme une autre, qui peut être stockée, copiée, transmise sur un réseau, ou même **lue et exécutée par un autre programme** (un interpréteur ou un compilateur).

Exemple. Un programme peut être représenté comme une simple liste d'instructions (une donnée), puis exécuté par un **interpréteur** qui la parcourt :

```
1 def interpreter(programme, accumulateur=0):
2     for instruction, valeur in programme:
3         if instruction == "ADD":
4             accumulateur += valeur
5         elif instruction == "MUL":
6             accumulateur *= valeur
7     return accumulateur
```

◆ **PROPRIÉTÉ Calculabilité indépendante du langage** Ce qu'un programme peut calculer ne dépend pas du **langage** dans lequel il est écrit : un algorithme calculable en Python l'est tout autant en C, en OCaml ou dans n'importe quel autre langage de programmation à usage général. Un langage peut rendre un calcul plus simple ou plus rapide à écrire, mais il ne rend jamais calculable ce qui ne l'était pas dans un autre langage.

DÉFINITION 2

Le **problème de l'arrêt** consiste à se demander s'il existe un programme capable de décider, pour **n'importe quel** programme P et **n'importe quelle** entrée e , si l'exécution de P sur e s'arrête ou boucle indéfiniment. Ce problème est **indécidable** : aucun programme ne peut le résoudre dans tous les cas.

◆ **PROPRIÉTÉ Argument intuitif d'indécidabilité** Supposons qu'existe une fonction $s_arrete(P, e)$ qui renvoie toujours correctement True si le programme P s'arrête sur l'entrée e , et False sinon. Construisons alors un programme paradoxe qui, exécuté sur lui-même comme entrée, fait le **contraire** de ce que prédit s_arrete :

- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut True (prédit que paradoxe s'arrête), alors paradoxe boucle volontairement à l'infini ;
- ▶ si $s_arrete(paradoxe, paradoxe)$ vaut False (prédit que paradoxe boucle), alors paradoxe s'arrête immédiatement.

Dans les deux cas, `s_arrete` se trompe sur paradoxe appliqué à lui-même : une telle fonction `s_arrete` ne peut donc pas exister pour **tous** les cas.

⚠ ERREUR FRÉQUENTE

Croire que l'indécidabilité du problème de l'arrêt signifie qu'on ne peut jamais savoir si un programme donné s'arrête. Ce n'est pas ce que dit le résultat : on peut très bien prouver, au cas par cas, que tel ou tel programme particulier s'arrête (ou boucle). Ce qui est impossible, c'est d'écrire un **unique** programme qui répondrait correctement pour **tous** les programmes et toutes les entrées possibles, sans exception.

5.2 Écrire et analyser un programme récursif

DÉFINITION 1

Un programme **récursif** est une fonction qui s'appelle elle-même, directement ou indirectement. Il comporte toujours au moins un **cas de base** (qui s'arrête sans appel récursif) et un **cas général** (qui se ramène à une instance plus simple du même problème).

Exemple. La factorielle, un exemple classique dans un domaine mathématique :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

Exemple. Compter les occurrences d'un caractère dans une chaîne, dans un domaine différent (traitement de texte) :

```
1 def compter(chaine, caractere):
2     if chaine == "":
3         return 0
4     premier = 1 if chaine[0] == caractere else 0
5     return premier + compter(chaine[1:], caractere)
```

DÉFINITION 2

Analyser un programme récursif consiste à comprendre son **arbre d'appels** (quels appels déclenchent quels autres appels), à vérifier sa **terminaison** (le cas de base est-il toujours atteint ?), et à évaluer le nombre d'appels effectués.

Exemple. La version naïve du calcul de Fibonacci effectue un très grand nombre d'appels redondants : on peut le mesurer en instrumentant le code avec un compteur.

```
1 compteur_appels = 0
2
3 def fib_naif(n):
4     global compteur_appels
5     compteur_appels += 1
6     if n <= 1:
7         return n
```

```
8 return fib_naif(n - 1) + fib_naif(n - 2)
```

◆ **PROPRIÉTÉ Lecture de l'arbre d'appels** L'appel `fib_naif(4)` déclenche `fib_naif(3)` et `fib_naif(2)` ; `fib_naif(3)` déclenche à son tour `fib_naif(2)` et `fib_naif(1)`. On observe que `fib_naif(2)` est appelé **deux fois indépendamment**, avec le même résultat recalculé à chaque fois : c'est cette redondance, visible sur l'arbre d'appels, qui explique la croissance très rapide du nombre d'appels (voir le chapitre *Algorithmique* pour la solution par mémoïsation).

⚠ ERREUR FRÉQUENTE

Confondre « la fonction s'exécute plusieurs fois » et « la fonction boucle indéfiniment ». Un grand nombre d'appels récursifs (comme pour `fib_naif`) n'est pas un signe de non-terminaison : chaque branche de l'arbre d'appels atteint bien le cas de base ($n \leq 1$), la fonction termine toujours, mais avec un coût qui peut devenir très élevé.

5.3 API, bibliothèques et modules

DÉFINITION 1

Une **bibliothèque** (ou *librairie*) est un ensemble de fonctions déjà écrites et testées, réutilisables dans d'autres programmes. Son **API** (*Application Programming Interface*) est l'ensemble des fonctions, classes et paramètres qu'elle expose à l'utilisateur, **sans qu'il ait besoin de connaître leur implémentation interne**.

Exemple. La bibliothèque standard `statistics` fournit des fonctions déjà écrites et testées :

```
1 import statistics
2
3 notes = [12, 15, 8, 17, 14]
4 moyenne = statistics.mean(notes)
5 mediane = statistics.median(notes)
6 ecart_type = statistics.stdev(notes)
```

◆ **PROPRIÉTÉ Exploiter la documentation** Avant d'utiliser une fonction d'une bibliothèque, on consulte sa **documentation** : le nom et l'ordre des paramètres, leur type attendu, la valeur renvoyée, et des exemples d'utilisation. En Python, `help(fonction)` affiche la **docstring** (chaîne de documentation) associée à une fonction.

DÉFINITION 2

Un **module** est un fichier Python (`.py`) regroupant des fonctions liées par un thème commun, réutilisable dans d'autres programmes via `import`. Chaque fonction d'un module bien conçu est **documentée** par une docstring décrivant son rôle, ses paramètres et sa valeur de retour.

Exemple. Un module `geometrie.py` regroupant des fonctions sur les figures planes, chacune documentée :

```
1 def aire_rectangle(largeur, hauteur):
```

```

2     """Renvoie l'aire d'un rectangle.
3
4     Parametres : largeur, hauteur (nombres positifs).
5     Renvoie : l'aire (largeur * hauteur).
6     """
7     return largeur * hauteur
8
9 def perimetre_rectangle(largeur, hauteur):
10     """Renvoie le perimetre d'un rectangle."""
11     return 2 * (largeur + hauteur)

```

⚠ ERREUR FRÉQUENTE

Documenter le « comment » au lieu du « quoi ». Une bonne docstring décrit **ce que fait** la fonction, ses paramètres attendus et sa valeur de retour – pas le détail de son implémentation interne (qui peut changer sans que la documentation ait besoin d’être mise à jour, tant que le comportement observable reste le même).

5.4 Paradigmes de programmation

DÉFINITION 1

Un **paradigme de programmation** est une façon générale de structurer un programme et de raisonner sur son fonctionnement. Un même langage, et même un même programme, peut mobiliser **plusieurs paradigmes**.

Exemple. Calculer la somme des carrés des nombres pairs d’une liste, dans trois styles différents.

Style impératif (une suite d’instructions qui modifient un état) :

```

1 def somme_carres_pairs_imperatif(liste):
2     total = 0
3     for x in liste:
4         if x % 2 == 0:
5             total += x ** 2
6     return total

```

Style fonctionnel (composition de fonctions, sans modifier d’état) :

```

1 def somme_carres_pairs_fonctionnel(liste):
2     return sum(x ** 2 for x in liste if x % 2 == 0)

```

Style objet (les données et les traitements sont regroupés dans une classe) :

```

1 class ListeNombres:
2     def __init__(self, valeurs):
3         self.valeurs = valeurs
4
5     def somme_carres_pairs(self):
6         return sum(x ** 2 for x in self.valeurs if x % 2 == 0)

```

◆ PROPRIÉTÉ Choisir un paradigme selon le contexte

- ▶ Le style **impératif** est naturel pour décrire une suite d'étapes avec un état qui évolue (simulation, algorithme pas à pas).
- ▶ Le style **fonctionnel** favorise la lisibilité et limite les effets de bord pour des transformations de données (filtrer, transformer, agréger).
- ▶ Le style **objet** est adapté quand on manipule des **entités** avec un état propre et des comportements associés (une classe Compte, Pile, Noeud...).

Ce choix n'est presque jamais exclusif : un même projet combine typiquement les trois styles selon les besoins de chaque partie.

⚠ ERREUR FRÉQUENTE

Croire qu'un langage « impose » un seul paradigme. Python permet d'écrire du code impératif, fonctionnel (fonctions map, filter, expressions génératrices) et objet (classes) au sein d'un même programme, voire d'une même fonction. Le paradigme est un choix de conception, pas une contrainte du langage.

5.5 Mise au point : causes typiques de bugs

DÉFINITION 1

La **mise au point** (*débogage*) consiste à localiser et corriger les causes d'un comportement incorrect d'un programme. Certaines causes de bugs reviennent très fréquemment : erreurs liées aux **nombre flottants**, aux **effets de bord**, aux **inégalités**, et au **nommage**.

Exemple. Les nombres flottants ne représentent pas exactement toutes les valeurs décimales :

```
1 resultat = 0.1 + 0.2
2 print(resultat == 0.3)           # False, a priori surprenant
3 print(abs(resultat - 0.3) < 1e-9) # True : comparaison a une tolerance pres
```

Exemple. Un **effet de bord** inattendu par aliasing : deux noms qui désignent le même objet mutable.

```
1 def vider(liste):
2     liste.clear()
3
4 a = [1, 2, 3]
5 b = a           # b designe le MEME objet liste que a, pas une copie
6 vider(b)
```

⚠ ERREUR FRÉQUENTE

Croire que `b = a` crée une copie de la liste `a`. En Python, cette instruction fait seulement pointer `b` vers le **même objet** que `a` : modifier `b` (par une méthode qui modifie en place, comme `clear`, `append`, `sort`) modifie donc aussi `a`. Pour obtenir une copie indépendante, il faut écrire explicitement `b = a.copy()` ou `b = list(a)`.

Exemple. Une erreur d'**inégalité stricte/large** (*off-by-one*) très fréquente dans une boucle :

```
1 def indices_valides(liste, indice):
2     # Version correcte : indice doit verifier 0 <= indice < len(liste)
```

```
3 return 0 <= indice < len(liste)
```

Exemple. Une erreur de **nommage** : masquer (*shadow*) une fonction native de Python.

```
1 def somme_avec_bug(valeurs):
2     sum = 0           # masque la fonction native sum() !
3     for v in valeurs:
4         sum += v
5     return sum        # fonctionne ici, mais sum() natif est devenu inaccessible
```

⚠ ERREUR FRÉQUENTE

Nommer une variable comme une fonction native (`sum`, `list`, `type`, `id`...). Python l'autorise sans erreur, mais la fonction native devient **inaccessible sous ce nom** dans le reste du bloc de code : si l'on a besoin d'appeler `sum(...)` plus loin dans la même fonction, l'appel échouera avec une erreur de type « `int object is not callable` » puisque `sum` désigne maintenant un entier.

🔗 MÉTHODE M1 Écrire une fonction récursive et dérouler son arbre d'appels

Quand l'utiliser ? Dans les deux sens : quand on te demande d'écrire une fonction récursive sur un problème neuf, et quand on te donne une fonction récursive et qu'on demande ce qu'elle renvoie, combien d'appels elle fait, ou si elle termine. Écrire et dérouler sont la même démarche prise par les deux bouts.

Pas à pas — écrire :

1. **Écris le cas de base en premier**, avant toute autre ligne. Quelle est la plus petite entrée pour laquelle tu connais la réponse sans réfléchir ? La liste vide, la chaîne vide, $n = 0$.
2. **Écris comment le cas général se RAMÈNE à plus petit**. Une seule règle compte : l'argument de l'appel récursif doit être **strictement** plus proche du cas de base. Note cette quantité qui décroît — c'est ta preuve de terminaison.
3. **Suppose l'appel récursif correct** et écris la ligne qui combine son résultat avec le travail du niveau courant. C'est la ligne la plus courte et la plus difficile.
4. **Relis la terminaison** : est-ce que tout chemin d'exécution atteint le cas de base ? Une branche qui rappelle la fonction avec le même argument suffit à tout casser.

Pas à pas — dérouler :

1. **Écris l'appel initial**, puis, sous lui et décalés vers la droite, les appels qu'il déclenche, dans l'ordre du code.
2. **Descends jusqu'aux cas de base** : ce sont les feuilles de l'arbre, les seules qui renvoient sans rappeler.
3. **Remonte en écrivant les valeurs de retour**, feuille par feuille, en appliquant la ligne de combinaison.
4. **Compte**. Le nombre d'appels est le nombre de nœuds de l'arbre. Repère si un même argument apparaît dans deux branches : c'est le signe qu'une mémoïsation ferait gagner (voir la fiche du chapitre Algorithmique).

Exemple : `somme_chiffres(n)`, somme des chiffres d'un entier positif.

Cas de base : si $n < 10$, la somme est n lui-même.

Réduction : n vaut $n // 10$ avec un chiffre en moins ; la quantité qui décroît strictement est le nombre de chiffres.

Combinaison : $n \% 10 + \text{somme_chiffres}(n // 10)$.

Déroulé de `somme_chiffres(507)` : $7 + s(50)$, puis $0 + s(5)$, puis 5 — cas de base. On remonte : 5 , puis $0 + 5 = 5$, puis $7 + 5 = 12$. Trois appels, un seul par niveau : l'arbre est une chaîne, aucune mémoïsation à espérer.

Pièges :

- ▶ **Écrire le cas général avant le cas de base.** C'est l'ordre qui fait oublier le cas de base, et sans lui rien ne s'arrête.
- ▶ **Un appel récursif qui ne réduit rien.** $f(n)$ qui rappelle $f(n)$ ne termine jamais, même si le cas de base existe. Nomme toujours la quantité qui décroît.
- ▶ **Confondre « beaucoup d'appels » et « ne termine pas ».** $\text{fib}(30)$ fait plus de deux millions d'appels et termine parfaitement. Le problème est le coût, pas la terminaison, et la réponse n'est pas la même.
- ▶ **Dérouler dans le désordre.** L'arbre d'appels se lit dans l'ordre du code : l'appel écrit à gauche se déclenche en premier, et sa valeur revient en premier.

Vérifier : teste ta fonction sur le cas de base, sur le cas juste au-dessus, et sur un cas moyen comparé à une méthode directe. Pour un déroulé, compte les feuilles : elles doivent toutes être des cas de base, et aucun nœud interne ne doit avoir la même valeur de retour qu'un de ses enfants sans qu'une raison l'explique.

S'entraîner : exercices C4 et C5 du chapitre.

Exercice 1 ◆

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres d'un entier positif n (par exemple, $1234 \rightarrow 1 + 2 + 3 + 4 = 10$). Quel est le cas de base ? Combien d'appels récursifs sont nécessaires pour `somme_chiffres(1234)` ?

Exercice 2 ◆

En consultant la documentation du module standard `math` (par exemple avec `help(math.hypot)` et `help(math.gcd)`) :

1. Utiliser `math.hypot(x, y)` pour calculer l'hypoténuse d'un triangle rectangle de côtés 3 et 4.
2. Utiliser `math.gcd(a, b)` pour calculer le plus grand diviseur commun de 48 et 18.
3. La documentation précise que `math.sqrt(x)` attend un nombre. Que se passe-t-il si l'on appelle `math.sqrt("4")` (une chaîne de caractères) au lieu de `math.sqrt(4)` ? Vérifier en lisant le message d'erreur obtenu.

Exercice 3 ◆◆

La fonction suivante, écrite en style **impératif**, sélectionne et met en majuscules les mots d'une liste dont la longueur atteint un minimum donné :

```
1 def mots_longs_imperatif(mots, longueur_min):
2     resultat = []
3     for mot in mots:
4         if len(mot) >= longueur_min:
5             resultat.append(mot.upper())
6     return resultat
```

1. Réécrire cette fonction en style **fonctionnel**, sous le nom `mots_longs_fonctionnel`, en une seule expression (liste en compréhension).
2. Vérifier que les deux versions renvoient le même résultat sur `["chat", "ordinateur", "cle", "programmation"]` avec `longueur_min=6`.

Exercice 4 ◆

Le code suivant contient un bug lié à un **effet de bord** inattendu :

```

1 def ajouter_note(bulletin, note):
2     bulletin.append(note)
3     return bulletin
4
5 bulletin_A = []
6 bulletin_B = ajouter_note(bulletin_A, 15)
7 bulletin_C = ajouter_note(bulletin_A, 12)

```

1. Que contiennent `bulletin_A`, `bulletin_B` et `bulletin_C` après ces trois lignes ? Le résultat est-il celui attendu si l'on voulait deux bulletins **indépendants** (un avec la note 15, un autre avec la note 12) ?
2. Corriger `ajouter_note` pour qu'elle renvoie un **nouveau** bulletin sans modifier celui passé en argument.

LANGAGES, RÉCURSIVITÉ ET PARADIGMES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Affirmer qu'un programme est aussi une donnée signifie que :
 - A. un programme peut être lu, transformé ou exécuté par un autre programme.
 - B. un programme n'a aucun effet lorsqu'on l'exécute.
 - C. les programmes et les données occupent la même quantité de mémoire.
 - D. un programme ne peut être écrit que par une machine.

Capacité C2

2. [Q2] Dire que la calculabilité ne dépend pas du langage employé signifie que :
 - A. tous les langages s'exécutent à la même vitesse.
 - B. ce qui est calculable dans un langage l'est dans tout autre langage universel.
 - C. tous les langages ont la même syntaxe.
 - D. un algorithme ne peut s'écrire que dans un seul langage.

Capacité C3

3. [Q3] Dire que le problème de l'arrêt est indécidable signifie que :
 - A. aucun programme ne s'arrête jamais.
 - B. on ne peut jamais prouver qu'un programme donné s'arrête.
 - C. aucun programme unique ne peut décider, pour tout programme et toute entrée, si l'exécution s'arrête.
 - D. cette difficulté ne concerne que Python.

Capacité C4

4. [Q4] Dans une fonction récursive, le cas de base sert à :
 - A. accélérer le calcul.
 - B. déclarer les variables locales.
 - C. vérifier le type des arguments reçus.
 - D. arrêter la récursion en fournissant une valeur sans nouvel appel.

Capacité C5

5. [Q5] Un grand nombre d'appels dans l'arbre d'appels d'une fonction récursive indique que :
 - A. la fonction termine, mais peut coûter cher en temps.
 - B. le code comporte nécessairement une erreur.
 - C. la fonction ne termine jamais.
 - D. le cas de base est mal écrit.

Capacité C6

6. [Q6] L'API d'une bibliothèque désigne :
- A. son code source complet.
 - B. l'ensemble des fonctions et des paramètres qu'elle met à disposition.
 - C. le nom de son auteur.
 - D. sa vitesse d'exécution.

Capacité C7

7. [Q7] La documentation d'une fonction indique qu'elle « renvoie None si la clé est absente ». Le programme appelant doit :
- A. modifier la bibliothèque pour qu'elle lève une exception.
 - B. ignorer cette mention, qui ne concerne que les développeurs de la bibliothèque.
 - C. traiter explicitement ce cas avant d'utiliser la valeur renvoyée.
 - D. supposer que la clé est toujours présente.

Capacité C8

8. [Q8] Dans un module Python destiné à être réutilisé, le bloc `if __name__ == "__main__":` sert à :
- A. déclarer les fonctions exportées par le module.
 - B. documenter le module.
 - C. importer automatiquement toutes les dépendances.
 - D. n'exécuter le code de démonstration ou de test que si le fichier est lancé directement.

Capacité C9

9. [Q9] Un même programme Python peut mobiliser :
- A. plusieurs paradigmes à la fois : impératif, fonctionnel et objet.
 - B. le seul paradigme objet.
 - C. un seul paradigme, imposé par le langage.
 - D. aucun paradigme identifiable.

Capacité C10

10. [Q10] Pour appliquer une même transformation à chaque élément d'une liste, sans modifier la liste de départ, le style le mieux adapté est :
- A. impératif, en modifiant la liste sur place case par case.
 - B. fonctionnel, par une compréhension ou une application de fonction produisant une nouvelle liste.
 - C. objet, en créant une classe par élément.
 - D. aucun : cette transformation est impossible.

Capacité C11

11. [Q11] Comparer deux nombres flottants avec l'opérateur `==` est une source de bugs parce que :
- A. Python interdit cette comparaison.
 - B. l'opérateur ne fonctionne qu'entre entiers.
 - C. les flottants ne représentent pas exactement toutes les valeurs décimales.
 - D. tous les flottants sont considérés comme égaux entre eux.
12. [Q12] Écrire `b = a`, où `a` est une liste :
- A. vide la liste `a`.
 - B. crée une copie indépendante de `a`.
 - C. provoque une erreur.
 - D. fait désigner à `b` le même objet que `a`.



6 Structures de données

6.1 Interface et implémentation d'une structure de données

DÉFINITION 1

L'**interface** d'une structure de données est l'ensemble des **opérations** qu'elle propose (ce que l'on peut faire), **sans préciser comment** elles sont réalisées. Une **implémentation** est une façon concrète de coder ces opérations (avec quelles données internes, quel algorithme).

EXEMPLE Interface d'une pile (Stack) : empiler(valeur), depiler(), est_vide().

Implémentation 1 — avec une liste Python (fin de liste = sommet) :

```
1 class PileListe:
2     def __init__(self):
3         self._donnees = []
4
5     def empiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def depiler(self):
9         return self._donnees.pop()
10
11    def est_vide(self):
12        return len(self._donnees) == 0
```

Implémentation 2 — avec une liste chaînée (debut = sommet) :

```
1 class Maillon:
2     def __init__(self, valeur, suivant=None):
3         self.valeur = valeur
4         self.suivant = suivant
5
6 class PileChaînée:
7     def __init__(self):
8         self._sommet = None
9
10    def empiler(self, valeur):
11        self._sommet = Maillon(valeur, self._sommet)
12
13    def depiler(self):
14        v = self._sommet.valeur
15        self._sommet = self._sommet.suivant
16        return v
17
18    def est_vide(self):
19        return self._sommet is None
```

EXEMPLE Ces deux implementations offrent **exactement la même interface** (mêmes trois méthodes, même comportement observable), mais des structures de données internes totalement différentes (tableau dynamique contre liste chaînée).

⚠ ERREUR FRÉQUENTE

Confondre interface et implementation lors d'un test. Un test doit porter sur le comportement observable via l'interface (par exemple : après avoir empiler 1, 2, 3, dépiler doit renvoyer 3), pas sur les détails internes (comme la structure exacte de `_donnees`). Cela permet au même jeu de tests de valider plusieurs implementations.

6.2 Définir et utiliser une classe en Python

DÉFINITION 1

Une **classe** est un modèle qui décrit la structure (**attributs**, l'état) et le comportement (**méthodes**) des objets qui en sont des **instances**. En Python, une classe se définit avec le mot-cle `class`, et le constructeur `__init__` initialise les attributs d'une nouvelle instance.

Exemple.

```
1 class Point:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def distance_origine(self):
7         return (self.x ** 2 + self.y ** 2) ** 0.5
8
9     def translater(self, dx, dy):
10        self.x = self.x + dx
11        self.y = self.y + dy
```

Utilisation :

```
1 p = Point(3, 4)
2 print(p.x, p.y)           # acces aux attributs
3 print(p.distance_origine()) # appel de methode : 5.0
4 p.translater(1, 1)
5 print(p.x, p.y)           # 4 5
```

- ◆ **PROPRIÉTÉ Vocabulaire**
 - **Attribut** : donnée associée à un objet (état), accédée par objet.attribut.
 - **Méthode** : fonction associée à une classe, appelée par objet.methode(...). Le premier paramètre `self` désigne l'objet lui-même (fourni automatiquement lors de l'appel).
 - **Instance** : un objet particulier créé à partir d'une classe (par exemple `p` est une instance de `Point`).

ERREUR FRÉQUENTE

Oublier self dans la définition d'une méthode. Toute méthode d'instance doit avoir self comme premier paramètre (même si on ne l'appelle jamais explicitement lors de l'appel objet.methode()) : Python le fournit automatiquement.

6.3 Piles, files, et choix de structure adaptée

DÉFINITION 1

Une **pile** (Stack) fonctionne selon le principe **LIFO** (*Last In, First Out*) : le dernier élément ajouté est le premier retire. Interface : empiler, depiler, est_vide.

Une **file** (Queue) fonctionne selon le principe **FIFO** (*First In, First Out*) : le premier élément ajouté est le premier retire. Interface : enfiler, defiler, est_vide.

Exemple. Implementation d'une file avec une liste Python (collections.deque est préférable en pratique pour l'efficacité, mais une liste suffit pour illustrer le principe) :

```

1 class File:
2     def __init__(self):
3         self._donnees = []
4
5     def enfiler(self, valeur):
6         self._donnees.append(valeur)
7
8     def defiler(self):
9         return self._donnees.pop(0)
10
11    def est_vide(self):
12        return len(self._donnees) == 0

```

Ici, "A" est le premier entre et le premier sorti : comportement FIFO, à l'opposé du comportement LIFO d'une pile.

◆ **PROPRIÉTÉ Choisir une structure adaptée** Le choix dépend des **opérations dominantes** de la situation à modéliser :

- ▶ **Pile** : annulation d'actions (undo), évaluation d'expressions, parcours en profondeur.
- ▶ **File** : gestion de tâches dans l'ordre d'arrivée, parcours en largeur, files d'attente.
- ▶ **Liste** : accès par position (indice), parcours séquentiel.
- ▶ **Dictionnaire** : accès direct par cle (recherche rapide en moyenne, indépendante de la taille), sans notion d'ordre de position.

ERREUR FRÉQUENTE

Confondre .pop() et .pop(0). Pour une liste Python, .pop() retire et renvoie le **dernier** élément (comportement LIFO, utile pour une pile), tandis que .pop(0) retire et renvoie le **premier** élément (comportement FIFO, utile pour une file, mais coûteux car il faut décaler tous les éléments restants).

6.4 Arbres binaires

DÉFINITION 1

Un **arbre binaire** est soit **vide**, soit constitué d'une **racine** (une valeur) et de deux **sous-arbres** (gauche et droit), eux-mêmes des arbres binaires. Une **feuille** est un noeud sans sous-arbre (les deux sous-arbres sont vides).

- ◆ **PROPRIÉTÉ Mesures usuelles**
- La **taille** d'un arbre est son nombre total de noeuds.
 - La **hauteur** d'un arbre est la longueur du plus long chemin de la racine à une feuille (hauteur 0 pour un arbre réduit à une seule feuille, hauteur -1 ou non définie pour un arbre vide selon la convention retenue).
 - Le nombre de **feuilles** est le nombre de noeuds sans sous-arbre.

Exemple. Implementation d'un arbre binaire et calcul de ses mesures :

```

1 class Arbre:
2     def __init__(self, valeur, gauche=None, droit=None):
3         self.valeur = valeur
4         self.gauche = gauche
5         self.droit = droit
6
7     def taille(a):
8         if a is None:
9             return 0
10        return 1 + taille(a.gauche) + taille(a.droit)
11
12    def hauteur(a):
13        if a is None:
14            return -1
15        return 1 + max(hauteur(a.gauche), hauteur(a.droit))
16
17    def nb_feuilles(a):
18        if a is None:
19            return 0
20        if a.gauche is None and a.droit is None:
21            return 1
22        return nb_feuilles(a.gauche) + nb_feuilles(a.droit)

```

Pour l'arbre $(1, (2, (4, \emptyset, \emptyset), \emptyset), (3, \emptyset, \emptyset))$ (racine 1, fils gauche 2 ayant lui-même un fils gauche 4, fils droit 3) :

taille = 4, hauteur = 2 (chemin $1 \rightarrow 2 \rightarrow 4$), nombre de feuilles = 2 (les noeuds 4 et 3).

⚠ ERREUR FRÉQUENTE

Oublier le cas de base dans une fonction recursive sur un arbre. Toute fonction recursive sur un arbre binaire doit traiter explicitement le cas `a is None` (arbre vide) en premier, sous peine d'erreur (`AttributeError`) lorsque la recursion atteint un sous-arbre vide.

6.5 Graphes : modélisation et représentations

DÉFINITION 1

Un **graphe** est constitué de **sommets** (ou noeuds) reliés par des **arêtes**. Un graphe est **orienté** si ses arêtes ont un sens (representees par des fleches), **non orienté** sinon.

◆ **PROPRIÉTÉ Représentation par matrice d'adjacence** Pour un graphe a n sommets numerotes de 0 a $n - 1$, la **matrice d'adjacence** est un tableau $n \times n$ ou la case (i, j) vaut 1 (ou True) s'il existe une arête de i vers j , 0 sinon.

Exemple. Graphe orienté a 3 sommets : $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$.

```
1 matrice = [
2     [0, 1, 1],
3     [0, 0, 1],
4     [0, 0, 0],
5 ]
```

◆ **PROPRIÉTÉ Représentation par listes de successeurs** Pour chaque sommet, on stocke la **liste des sommets accessibles directement** (ses successeurs), par exemple sous forme de dictionnaire.

Exemple. Le même graphe ($0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 2$) représente par listes de successeurs :

```
1 successeurs = {
2     0: [1, 2],
3     1: [2],
4     2: [],
5 }
```

Conversion. Pour passer de la matrice aux listes de successeurs, on parcourt chaque ligne i et on retient les indices j ou $\text{matrice}[i][j] == 1$:

```
1 def matrice_vers_listes(matrice):
2     n = len(matrice)
3     return {i: [j for j in range(n) if matrice[i][j] == 1] for i in range(n)}
```

◆ **PROPRIÉTÉ Comparaison** La **matrice d'adjacence** permet de tester en temps constant l'existence d'une arête (i, j) , mais occupe une memoire en n^2 même si le graphe est peu dense (**creux**). Les **listes de successeurs** occupent une memoire proportionnelle au nombre d'arêtes reellement presentes, plus adaptees aux graphes creux, mais tester l'existence d'une arête précise peut necessiter de parcourir une liste.

⚠ ERREUR FRÉQUENTE

Oublier qu'un sommet sans successeur doit quand même apparaitre. Dans la représentation par listes de successeurs, un sommet isole ou puits (sans arête sortante) doit être present dans

le dictionnaire avec une liste **vide**, pas absent du dictionnaire : sinon, un parcours du graphe risque d'ignorer ce sommet.

🔗 MÉTHODE M1 Spécifier une structure de données par son interface

Quand l'utiliser ? Quand l'énoncé décrit un besoin en français — « on veut gérer une file d'attente de patients », « on veut un annuaire consultable » — et demande de **spécifier** la structure avant de l'implémenter. Spécifier, c'est dire ce que la structure fait, jamais comment elle le fait.

Pas à pas :

1. **Liste les opérations** que le texte demande, une par verbe : ajouter, retirer, consulter, compter, tester la vacuité. Si un verbe n'apparaît pas dans l'énoncé, ne l'invente pas : une interface minimale se défend mieux qu'une interface bavarde.
2. **Donne à chacune sa signature** : son nom, ses paramètres, et ce qu'elle renvoie. Distingue nettement celles qui **renvoient** une valeur de celles qui **modifient** la structure.
3. **Écris ce que chaque opération exige** — sa précondition. Dépiler exige une pile non vide ; consulter le premier exige au moins un élément.
4. **Écris ce que chaque opération garantit** — sa postcondition, en reliant l'état d'après à l'état d'avant. Après empiler(v), la structure contient un élément de plus et son sommet vaut v.
5. **Décide des cas limites** explicitement : structure vide, un seul élément, élément absent. Une spécification qui ne dit rien du cas vide est incomplète.
6. **Relis en interdisant tout mot d'implémentation**. Si ta spécification parle de liste, d'indice, de maillon ou de append, elle a déjà choisi l'implémentation : réécris-la.

Exemple : « on veut gérer une file d'attente de patients : on inscrit un patient, on appelle le suivant, et on veut savoir s'il reste quelqu'un ».

Opérations : inscrire(patient), appeler(), est_vide().

Signatures : inscrire prend un patient et ne renvoie rien ; appeler ne prend rien et renvoie un patient ; est_vide ne prend rien et renvoie un booléen.

Préconditions : appeler exige une file non vide. Les deux autres n'exigent rien.

Postconditions : après inscrire(p), la file contient un patient de plus et est_vide() est faux. appeler renvoie le patient inscrit **le plus anciennement** et le retire.

C'est cette dernière phrase — le plus anciennement — qui fait de la structure une file et non une pile. Elle appartient à la spécification, pas à l'implémentation.

Pièges :

- ▶ **Décrire l'implémentation en croyant spécifier.** « On stocke dans une liste et on retire avec pop(0) » n'est pas une interface : c'est déjà un choix, et il ferme la porte à toute autre implémentation.
- ▶ **Oublier la précondition du cas vide.** C'est le seul cas où la structure ne peut pas répondre, et c'est celui que l'énoncé teste.
- ▶ **Confondre renvoyer et modifier.** Une opération qui consulte sans retirer et une opération qui retire sont deux opérations différentes, avec deux postconditions.
- ▶ **Une spécification qui ne distingue pas file et pile.** Si ton contrat est satisfait aussi bien par une pile que par une file, c'est qu'il ne dit pas dans quel ordre les éléments sortent — la phrase la plus importante manque.

Vérifier : écris ton contrat sous forme de jeu de tests, puis fais-le passer par **deux** implémentations aux données internes différentes. S'il passe les deux, il ne parle bien que de l'interface. S'il échoue sur l'une, c'est qu'il a déjà choisi.

S'entraîner : exercices C1 du chapitre.

④ MÉTHODE M2 Modéliser une situation par un graphe

Quand l'utiliser ? Quand l'énoncé décrit un réseau, des relations, des dépendances ou des déplacements, et demande de « modéliser la situation par un graphe ». La difficulté n'est pas de dessiner : elle est de décider ce que sont les sommets, et surtout de s'y tenir.

Pas à pas :

1. **Choisis les sommets** : ce sont les **objets** entre lesquels une relation existe. Écris la phrase « un sommet est un... ». Si tu n'arrives pas à finir la phrase avec un nom unique, ton choix est mauvais.
2. **Choisis les arêtes** : ce sont les **relations**. Écris la phrase « il y a une arête entre u et v lorsque... ». Cette phrase doit être vérifiable sur l'énoncé, sans interprétation.
3. **Décide de l'orientation**. La relation est-elle symétrique ? « être ami avec » l'est, « suivre sur un réseau social » ne l'est pas, « précéder dans un enchaînement de tâches » non plus. Un graphe orienté à tort double le nombre d'arêtes ; un graphe non orienté à tort efface l'information du sens.
4. **Décide de la pondération**. Si l'énoncé donne des durées, des distances ou des coûts, ils vont **sur les arêtes**. S'il n'en donne pas, ne pondère pas.
5. **Écris la représentation** : dictionnaire de listes de successeurs, ou matrice d'adjacence. Pour un graphe non orienté, chaque arête doit apparaître **deux fois**, une dans chaque sens.
6. **Relis la question posée**. Elle se traduit presque toujours en un terme du cours : « peut-on aller de A à B » est une recherche de chemin, « peut-on tout visiter » est une question de connexité, « y a-t-il une boucle de dépendances » est une détection de cycle.

Exemple : « Quatre stations de métro. La ligne 1 relie A et B , et B et C . La ligne 2 relie C et D . Peut-on aller de A à D ? »

Sommets : un sommet est une **station**. Quatre sommets : A, B, C, D .

Arêtes : il y a une arête entre deux stations lorsqu'une ligne les relie directement.

Orientation : le métro se parcourt dans les deux sens, la relation est symétrique : graphe **non orienté**.

Pondération : l'énoncé ne donne aucune durée : aucune pondération.

Représentation : $\{A : [B], B : [A, C], C : [B, D], D : [C]\}$ — chaque arête figure bien deux fois.

Traduction de la question : « peut-on aller de A à D » est une recherche de chemin. Un parcours depuis A atteint D : oui.

Pièges :

- **Prendre pour sommets ce que la question ne demande pas**. Ici, modéliser les *lignes* comme sommets donne un graphe correct mais qui ne peut pas répondre à une question portant sur des stations.
- **Orienter une relation symétrique**. Le modèle devient asymétrique : on peut aller de A à D mais plus de D à A , ce qu'aucun métro ne fait.
- **Symétriser une relation qui ne l'est pas**. « Alan suit Ada » ne veut pas dire « Ada suit Alan » : symétriser invente des trajets qui n'existent pas.
- **N'écrire l'arête que dans un sens** dans un dictionnaire censé être non orienté. Le parcours devient faux, et rien ne le signale.

Vérifier : pour un graphe non orienté, contrôle que chaque voisin te renvoie l'ascenseur — si v figure chez u , u doit figurer chez v . Puis relis les deux phrases de l'étape 1 et 2 à côté de l'énoncé : elles doivent se vérifier ligne à ligne.

S'entraîner : exercices C11 du chapitre.

④ MÉTHODE M3 Passer d'une représentation de graphe à l'autre

Quand l'utiliser ? Quand l'énoncé donne une matrice d'adjacence et demande les listes de successeurs, ou l'inverse, ou quand un algorithme fourni attend l'autre représentation que celle de l'énoncé. La conversion est mécanique, mais deux détails la font rater : l'ordre des sommets, et les sommets sans voisin.

Pas à pas — de la matrice vers les listes :

1. **Numérote les sommets** et écris la correspondance numéro $\leftrightarrow$ nom. La ligne i et la colonne i désignent le **même** sommet : si tu changes l'ordre en route, tout est faux.
2. **Parcours la ligne i** , de gauche à droite. Chaque 1 en colonne j donne un successeur j du sommet i .
3. **Écris la liste de la ligne i** , même vide. Un sommet sans successeur doit apparaître avec une liste **vide**, pas disparaître.
4. **Recommence pour chaque ligne**. Tu dois obtenir autant d'entrées que la matrice a de lignes.

Pas à pas — des listes vers la matrice :

1. **Compte les sommets** : ce sont les clés, plus tout sommet qui n'apparaît que comme voisin. Oublier ces derniers donne une matrice trop petite.
2. **Pose une matrice $n \times n$ remplie de 0**, sommets rangés dans l'ordre choisi à l'étape précédente.
3. **Pour chaque successeur v de u , mets un 1 en ligne u , colonne v** — dans cet ordre, jamais l'inverse.
4. **Si le graphe est non orienté**, mets aussi le 1 symétrique. La matrice d'un graphe non orienté est **symétrique** : c'est ton contrôle.

Exemple : graphe orienté à trois sommets, $0 \rightarrow 1$, $0 \rightarrow 2$, $1 \rightarrow 2$.

Matrice, lignes et colonnes dans l'ordre 0, 1, 2 :

ligne 0 : 0 1 1 ; ligne 1 : 0 0 1 ; ligne 2 : 0 0 0.

Listes : $\{0 : [1, 2], 1 : [2], 2 : []\}$. Le sommet 2 est un **puits** : sa liste est vide, et il figure quand même.

Retour aux listes vers la matrice : la ligne 2 ne contient que des 0, ce qui est cohérent. La matrice n'est **pas** symétrique, et c'est normal : le graphe est orienté.

Pièges :

- **Échanger ligne et colonne**. Mettre le 1 en ligne v , colonne u construit la **transposée** : toutes les arêtes sont inversées. Sur un graphe non orienté, l'erreur est invisible — la matrice est symétrique — et elle apparaîtra plus tard, sur un graphe orienté, sans que rien ne l'annonce.
- **Faire disparaître un sommet sans successeur**. Un puits doit rester dans le dictionnaire avec une liste vide, sinon un parcours l'ignore.
- **Oublier un sommet qui n'apparaît que comme voisin**. Il n'est clé de rien, mais il compte dans la taille de la matrice.
- **Changer l'ordre des sommets entre lignes et colonnes**. La correspondance numéro $\leftrightarrow$ nom doit être écrite une fois, et servir aux deux dimensions.

Vérifier : refais la conversion **dans l'autre sens** et compare au point de départ — les deux conversions sont réciproques. Et pour un graphe non orienté, contrôle que la matrice est symétrique : c'est immédiat à l'œil.

S'entraîner : exercices C14 du chapitre.

Exercice 1 ♦♦

On veut vérifier si une expression comportant des parenthèses $()$, crochets $[]$ et accolades $\{\}$ est **bien parenthesée** (chaque symbole ouvrant a un symbole fermant correspondant, dans le bon ordre).

1. Expliquer pourquoi une **pile** est une structure adaptée à ce problème (justifier par les opérations dominantes).
2. Écrire une fonction `est_equilibre(expr)` qui utilise une pile (interface `empiler`, `depiler`, `est_vide`) pour résoudre ce problème.
3. Tester votre fonction sur `"([{}])"` (bien parenthesée) et `"([)]"` (mal parenthesée).

Exercice 2 ◆

Écrire une classe `Compte` représentant un compte bancaire simplifié, avec :

- ▶ un constructeur prenant un titulaire et un solde initial (par défaut 0) ;
- ▶ une méthode `deposer(montant)` qui augmente le solde ;
- ▶ une méthode `retirer(montant)` qui diminue le solde, et lève une erreur (`ValueError`) si le montant demande dépasse le solde disponible.

Tester votre classe avec un compte initial de 100, un dépôt de 50, puis un retrait de 30.

Exercice 3 ◆◆

On construit un arbre binaire "en peigne" : la racine vaut 1, son fils gauche vaut 2 et a lui-même un fils gauche 3, qui a un fils gauche 4, qui a un fils gauche 5 (aucun fils droit nulle part).

1. En utilisant la classe `Arbre` du cours et les fonctions `taille` et `hauteur`, construire cet arbre en Python.
2. Sans exécuter de code, prédire la taille et la hauteur de cet arbre. Justifier.
3. Vérifier votre prédiction en appelant `taille` et `hauteur` sur l'arbre construit.

Exercice 4 ◆◆

On donne le graphe orienté à 4 sommets (0, 1, 2, 3) représenté par la matrice d'adjacence :

```

1 matrice = [
2     [0, 1, 0, 1],
3     [0, 0, 1, 0],
4     [0, 0, 0, 0],
5     [0, 0, 1, 0],
6 ]

```

1. Lister toutes les arêtes de ce graphe (sous la forme $i \rightarrow j$).
2. Le sommet 2 a-t-il des successeurs ? Interpréter.
3. En utilisant la fonction `matrice_vers_listes` du cours, donner la représentation par listes de successeurs de ce graphe.

STRUCTURES DE DONNÉES — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] L'interface d'une structure de données décrit :
 - A. les opérations disponibles, sans préciser comment elles sont réalisées.
 - B. la façon dont les données sont rangées en mémoire.
 - C. le nombre d'octets occupés.
 - D. le langage de programmation employé.

Capacité C2

2. [Q2] Passer d'une implémentation d'une pile par tableau à une implémentation par liste chaînée, sans changer son interface :
 - A. oblige à réécrire tous les programmes qui utilisent la pile.
 - B. laisse inchangés les programmes qui n'utilisent que `empiler` et `depiler`.
 - C. change le principe LIFO en principe FIFO.
 - D. est impossible : l'interface dépend de l'implémentation.

Capacité C3

3. [Q3] Une file peut être implémentée par une liste Python où l'on retire en tête avec `pop(0)`, ou par deux piles. Ces deux implémentations :
- A. respectent le même contrat, mais n'ont pas le même coût.
 - B. sont interdites : une file n'admet qu'une implémentation.
 - C. produisent des ordres de sortie différents.
 - D. ont exactement les mêmes performances.

Capacité C4

4. [Q4] Dans la définition d'une classe Python, la méthode chargée d'initialiser un nouvel objet se nomme :
- A. `init`
 - B. `__init__`
 - C. `create`
 - D. `new`

Capacité C5

5. [Q5] Dans une méthode de classe Python, le paramètre `self` désigne :
- A. le premier attribut déclaré.
 - B. la classe elle-même.
 - C. l'objet sur lequel la méthode est appelée.
 - D. rien de particulier : il est facultatif.

Capacité C6

6. [Q6] On empile successivement 1, puis 2, puis 3 dans une pile vide, puis on dépile une fois. La valeur obtenue est :
- A. 1
 - B. la plus petite des trois.
 - C. 2
 - D. 3

Capacité C7

7. [Q7] Pour mémoriser les pages visitées afin de revenir en arrière dans un navigateur, la structure adaptée est :
- A. une pile.
 - B. un arbre binaire de recherche.
 - C. une file.
 - D. une matrice d'adjacence.

Capacité C8

8. [Q8] Rechercher une valeur associée à une clé est en moyenne plus rapide dans un dictionnaire que dans une liste, parce que :
- A. un dictionnaire est toujours plus petit qu'une liste.
 - B. l'accès passe par une table de hachage, sans dépendre du nombre d'éléments.
 - C. l'affirmation est fausse : les deux coûtent autant.
 - D. les dictionnaires sont triés automatiquement.

Capacité C9

9. [Q9] Parmi ces situations, laquelle appelle le plus naturellement une structure arborescente ?
- A. La liste des élèves d'une classe, par ordre alphabétique.
 - B. La file d'attente des travaux d'impression.
 - C. L'arborescence des dossiers et fichiers d'un disque.
 - D. Le relevé des températures d'une journée, heure par heure.

Capacité C10

10. [Q10] La taille d'un arbre binaire est :
- A. la longueur du plus long chemin de la racine à une feuille.
 - B. le nombre de ses niveaux moins un.
 - C. son nombre de feuilles.

D. son nombre de nœuds.

Capacité C11

11. [Q11] Pour modéliser un réseau social où l'amitié est réciproque, on emploie :
- A. un graphe non orienté, les personnes étant les sommets et les amitiés les arêtes.
 - B. une pile de profils.
 - C. un arbre binaire de recherche.
 - D. un graphe orienté, car l'amitié a toujours un sens.

Capacité C12

12. [Q12] Dans la matrice d'adjacence d'un graphe à n sommets, la case (i, j) vaut 1 lorsque :
- A. les sommets i et j portent le même numéro.
 - B. il existe une arête reliant i à j .
 - C. le sommet i est une feuille.
 - D. la somme $i + j$ vaut n .

Capacité C13

13. [Q13] Dans une implémentation par listes de successeurs, l'espace occupé par un graphe à n sommets et m arêtes est de l'ordre de :
- A. n^2 , quel que soit m .
 - B. m seulement, sans dépendre de n .
 - C. $n + m$.
 - D. $n \times m$.

Capacité C14

14. [Q14] Pour convertir une matrice d'adjacence en listes de successeurs, il faut :
- A. conserver seulement la diagonale de la matrice.
 - B. trier les sommets par nombre de voisins.
 - C. recopier chaque ligne de la matrice telle quelle, zéros compris.
 - D. pour chaque sommet, retenir les indices des colonnes dont la case vaut 1.



7 Conduire un projet informatique

7.1 Conduire un projet informatique en Terminale

THÉORÈME Contrat horaire officiel

Le programme réserve **au moins 25 %** de l'horaire total de la spécialité à la conception et à l'élaboration de projets conduits par les élèves. Sur une référence de 216 heures (6 heures hebdomadaires pendant 36 semaines), cela représente au moins 54 heures. Si l'horaire effectivement assuré diffère, le plan annuel est ajusté pour préserver la proportion minimale d'un quart.

Programme d'enseignement et Définition de l'épreuve

Cette unité met en œuvre une obligation du **Programme d'enseignement** (MENE1921247A). Elle ne constitue pas une nouvelle partie de l'examen et **ne remplace pas l'épreuve pratique**. La **Définition de l'épreuve** relève d'une autorité distincte : la note de service MENE2516123N définit, à compter de la session 2026, une partie écrite de 3 h 30 et une partie pratique d'une heure, chacune notée sur 20, avec des poids respectifs de 0,75 et 0,25. Le projet prépare des compétences utiles, mais son volume, ses livrables et sa grille ci-dessous relèvent de l'enseignement et non du format de l'épreuve.

Prérequis

Avant le cadrage, chaque équipe vérifie qu'elle sait :

- ▶ écrire, exécuter et tester un programme Python ;
- ▶ mobiliser au moins une notion du programme de Terminale NSI ;
- ▶ conserver un historique des versions et attribuer chaque contribution ;
- ▶ partager uniquement des données licites, documentées et dépourvues de données personnelles d'élèves ;
- ▶ expliquer une décision technique à l'oral et par écrit.

Question de départ et périmètre

Le projet répond à une question formulée par l'équipe et validée avec le professeur. Le thème peut provenir d'une autre discipline ou mobiliser, par exemple, une base de données, un graphe, une structure complexe, un traitement d'image ou de son, un site Web, un objet connecté ou un module d'intelligence artificielle. Le choix reste compatible avec les ressources disponibles et une réalisation complète en 54 heures de référence.

Le cahier des charges distingue :

- ▶ un **produit minimal viable**, indispensable et démontrable ;
- ▶ des fonctionnalités souhaitables, réalisées seulement après validation du socle ;
- ▶ les limites connues, hypothèses, dépendances, licences et données utilisées ;
- ▶ les critères observables qui permettront d'accepter ou de refuser chaque fonction.

ERREUR FRÉQUENTE

Accumuler des fonctionnalités sans produit minimal testable. Le programme demande une ambition raisonnable : une fonctionnalité non indispensable est reportée si elle met en danger l'aboutissement, les tests ou la documentation.

Jalons communs — plan de référence de 54 heures

Chaque jalon se termine par un point d'étape avec le professeur. L'équipe présente une preuve vérifiable, consigne la décision et met à jour les objectifs.

Jalon	Travail attendu	Durée	Preuve de passage
J0	Problème, équipe, responsabilités, contraintes et risques	3 h	Note de cadrage validée
J1	Cahier des charges, cas d'usage et critères d'acceptation	6 h	Spécification versionnée
J2	Architecture, données, interfaces et prototype exploratoire	6 h	Schéma et prototype exécutable
J3	Développement du produit minimal viable	9 h	Démonstration du socle
J4	Deuxième itération et intégration des contributions	9 h	Version intégrée et journal de décisions
J5	Tests normaux, cas limites, correction et robustesse	9 h	Rapport de tests reproductible
J6	Documentation utilisateur et technique, licences, limites	6 h	Documentation relue
J7	Stabilisation, démonstration, oral et bilan individuel	6 h	Version finale et rétrospective
Total		54 h	

À chaque point d'étape, l'équipe répond à quatre questions : qu'est-ce qui fonctionne ? Quelle preuve le montre ? Quel obstacle reste prioritaire ? Quel objectif faut-il conserver, réduire ou redéfinir ? Une modification de périmètre n'efface jamais l'historique de la décision.

Livrables obligatoires

1. une note de cadrage et un cahier des charges datés ;
2. un découpage en tâches, des responsabilités et un calendrier de jalons ;
3. les sources exécutables avec historique des versions et instructions de lancement ;
4. un jeu de tests couvrant les fonctions centrales et des cas limites ;
5. une documentation utilisateur et une documentation technique ;
6. un registre des données, bibliothèques, licences, hypothèses et limites ;
7. une démonstration finale reproductible ;
8. un bilan individuel identifiant contributions, apprentissages et améliorations.

Grille d'évaluation

La grille est communiquée dès J0. L'évaluation croise une production collective et des preuves individuelles ; elle ne se réduit ni au volume de code ni à la seule démonstration finale.

Critère	Poids	Preuve principale
Problème, périmètre et critères d'acceptation	15 %	Cadrage et cahier des charges
Architecture et choix techniques justifiés	15 %	Schémas et journal de décisions
Exactitude et fonctionnement du produit	25 %	Version exécutable
Tests, cas limites et traitement des erreurs	20 %	Rapport de tests
Documentation, reproductibilité et provenance	10 %	Guides et registre des sources
Organisation, jalons et contributions individuelles	10 %	Historique et bilans
Démonstration et argumentation orale	5 %	Présentation finale
Total	100 %	

Un produit non exécutable ne peut pas obtenir les points de fonctionnement sur la base d'une simple intention. Inversement, un projet modeste mais complet, testé, documenté et argumenté satisfait mieux le contrat qu'un prototype ambitieux impossible à reproduire.

Dossiers de projets authentiques

Quatre dossiers types sont proposés pour guider les équipes dans la définition d'un périmètre équilibré, reliant les thèmes du programme de Terminale à un problème concret.

Dossier 1 — Système de gestion et consultation de données (SGBD et Web)

- **Problématique** : Concevoir une application de gestion et d'interrogation pour une médiathèque ou une association, avec persistance des données et interface de consultation.
- **Notions mobilisées** : Modèle relationnel, schéma relationnel (clés primaires et étrangères), requêtes SQL (SELECT, JOIN, agrégation, mises à jour), interface Web ou locale, séparation modèle/-vue.
- **Produit minimal viable (MVP)** : Base de données SQLite avec au moins 3 tables reliées, script d'initialisation, interface permettant la recherche multicritère et l'insertion contrôlée avec validation de saisie.
- **Extensions conditionnelles** : Authentification avec mot de passe haché, export de statistiques en format ouvert (CSV/JSON), pagination des résultats.
- **Critères de recette** : Intégrité référentielle garantie, requêtes paramétrées sans concaténation brute, tests unitaires sur les opérations CRUD élémentaires.

Dossier 2 — Planificateur d'itinéraires et optimisation sur graphes

- **Problématique** : Développer un outil d'aide à la mobilité calculant les itinéraires optimaux sur un réseau de transport ou routier.
- **Notions mobilisées** : Graphes orientés et pondérés (listes et dictionnaires d'adjacence), parcours en largeur (BFS), algorithme de Dijkstra, structures de données linéaires et files de priorité.
- **Produit minimal viable (MVP)** : Chargement d'un réseau depuis un fichier structuré, calcul du plus court chemin entre deux sommets, affichage textuel de l'itinéraire détaillé et du coût cumulé.
- **Extensions conditionnelles** : Prise en compte de contraintes horaires (correspondances), détection des nœuds critiques de coupure, rendu graphique du graphe.
- **Critères de recette** : Exactitude du chemin comparé à un oracle vérifié, gestion propre des graphes non connexes (détection d'absence de chemin), temps d'exécution maîtrisés.

Dossier 3 — Moteur d'indexation textuelle et compression par arbres

- **Problématique** : Réaliser un moteur de recherche local capable d'indexer une collection de documents textuels et de compresser les archives textuelles.
- **Notions mobilisées** : Arbres binaires de recherche (ABR), arbres de Huffman, parcours d'arbres (infixe, préfixe), dictionnaires Python, analyse fréquentielle, complexité algorithmique.

- ▶ **Produit minimal viable (MVP)** : Construction d'un index inversé reliant chaque mot aux documents sources, recherche par mot-clé avec score de pertinence, module de compression et décompression de Huffman sans perte.
- ▶ **Extensions conditionnelles** : Recherche booléenne combinatoire (ET / OU / SAUF), élimination des mots vides (stop-words), mesure comparée des taux de compression.
- ▶ **Critères de recette** : Décompression bit à bit rigoureusement identique à l'original (somme de contrôle vérifiée), recherche en temps logarithmique moyen sur les index arborescents.

Dossier 4 — Simulateur de réseau et protocoles de communication

- ▶ **Problématique** : Construire un simulateur d'échanges de messages entre plusieurs entités communicantes reliées par des routeurs virtuels.
- ▶ **Notions mobilisées** : Protocoles de routage (RIP / OSPF), architecture client-serveur avec sockets ou simulation à événements discrets, tables de routage, encapsulation des paquets, détection de pertes.
- ▶ **Produit minimal viable (MVP)** : Simulation de 4 à 6 routeurs interconnectés, calcul des routes par vecteur de distance (algorithme de Bellman-Ford / RIP simplifié), acheminement pas à pas d'un paquet.
- ▶ **Extensions conditionnelles** : Détection automatique de panne d'un lien et convergence vers une route alternative, gestion du décompte de sauts (TTL), simulation d'un protocole d'acquittement.
- ▶ **Critères de recette** : Détection des boucles de routage, convergence démontrée après modification de la topologie, journalisation horodatée de chaque transfert de paquet.

Bilan final

Le bilan collectif compare le résultat au cahier des charges. Le bilan individuel explicite une contribution précise, un problème rencontré, la méthode utilisée pour le résoudre et une amélioration possible. Les limites connues sont documentées ; elles ne sont ni masquées ni présentées comme des fonctionnalités achevées.

CONDUIRE UN PROJET INFORMATIQUE — QCM D'AUTO-ÉVALUATION

Pour chaque question, une seule réponse est exacte.

Capacité C1

1. [Q1] Le problème auquel répond un projet informatique doit être formulé :
 - A. de façon explicite, avec des critères permettant de constater qu'il est résolu.
 - B. de la manière la plus large possible, pour ne rien s'interdire.
 - C. seulement une fois le programme écrit.
 - D. par le professeur seul, et jamais par l'équipe.
2. [Q2] Dans un projet mené en équipe, répartir le travail suppose avant tout :
 - A. que chacun écrive le même code de son côté, pour comparer ensuite.
 - B. de définir qui fait quoi et comment les parties s'assemblent.
 - C. qu'une seule personne programme et que les autres observent.
 - D. de tirer les tâches au sort.

Capacité C2

3. [Q3] Un point d'étape est utile à condition de porter sur :
 - A. le nombre de lignes écrites depuis la dernière séance.
 - B. l'impression générale des membres de l'équipe.
 - C. des critères observables : ce qui fonctionne, ce qui est testé, ce qui reste à faire.
 - D. la seule date de rendu finale.
4. [Q4] À mi-parcours, l'équipe constate qu'une fonctionnalité prévue ne sera pas terminée. La conduite adaptée est :

- A. conserver le périmètre initial et rendre un projet inachevé.
- B. abandonner le projet et en commencer un autre.
- C. ajouter des membres à l'équipe pour rattraper le retard.
- D. réduire le périmètre en accord avec le professeur, et livrer un ensemble cohérent et testé.

Apprendre, s'entraîner, se tester, progresser : la collection Nexus

Réussite accompagne chaque élève jusqu'à la maîtrise.

- Un cours structuré en strates, avec la rubrique « À la machine » : chaque notion se reproduit au clavier.
- » Du code Python vérifié par exécution : aucune sortie de programme imprimée sans avoir tourné.
- ① Des méthodes pas à pas avec exemples résolus commentés et pièges à éviter.
- ♦♦ Trois parcours d'exercices gradués et chronométrés, avec coups de pouce.
- ↪ QCM diagnostiques, auto-évaluation par compétences et sujets aux formats officiels.
- ⊗ Des fiches de remédiation ciblées, reliées aux erreurs réellement diagnostiquées.